Lecture 10: Flow Matching

ODE 기반 생성의 새로운 패러다임 — 직선 경로와 Optimal Transport

목차 (Table of Contents)

- 0. 개요
  - 선수 지식
  - 학습 목표
  - 핵심 논문
  - 패러다임 비교 요약
- 1. Continuous Normalizing Flow (CNF) 복습
  - 1.1 강의 2의 Normalizing Flow와의 관계
  - 1.2 CNF의 문제점 (기존)
- 2. Flow Matching: 핵심 아이디어
  - 2.1 목표
  - 2.2 이상적인 학습 목표 (하지만 사용 불가)
  - 2.3 조건부 Flow Matching (Conditional FM) — 핵심 트릭
  - 2.4 Conditional Flow Matching Loss
- 3. 학습 & 생성 알고리즘
  - 3.1 학습
  - 3.2 생성 (Euler Method)
- 4. DDPM과의 수학적 관계
  - 4.1 경로 비교
  - 4.2 Gaussian Flow Matching ≈ Diffusion
  - 4.3 세 가지 예측 대상의 변환
- 5. 왜 Flow Matching이 더 좋은가?
  - 5.1 직선 경로의 장점
  - 5.2 벤치마크 비교
  - 5.3 SD 3에서의 실제 사용
- 6. Rectified Flow — 경로를 더 직선으로
  - 6.1 아이디어
  - 6.2 Reflow 절차
- 7. SDE 확장: Flow Matching + Diffusion
  - 7.1 MIT 6.S184의 통합 프레임워크
  - 7.2 가우시안 조건부 경로의 스코어
  - 7.3 Fokker-Planck 연결
- 8. Guidance in Flow Matching

    - 8.1 CFG in Flow Matching
    - 8.2 Guidance Distillation (FLUX)
- 9. Optimal Transport Conditional Flow Matching (OT-CFM)
    - 9.1 Mini-batch OT의 핵심 아이디어
    - 9.2 OT-CFM이 필요한 이유
- 10. 실습: Flow Matching vs DDPM 구현 비교
    - 10.1 학습 코드 비교
    - 10.2 생성 코드 비교
    - 10.3 완전한 Flow Matching 모델 구현
    - 10.4 OT-CFM (Optimal Transport Conditional Flow Matching) 구현
    - 10.5 고차 ODE 솔버: Heun's Method 및 RK4
    - 10.6 Flow Matching에서의 CFG 구현
- 11. 핵심 수식 요약
- 12. 다음 강의 미리보기
- 13. 참고문헌 (References)

---

0. 개요

선수 지식

이 강의를 이해하기 위해 다음 내용을 먼저 숙지해야 한다:

선수 과목	관련 강의	핵심 개념
Normalizing Flow	강의 2	Change of Variables, 가역 함수, 야코비안 행렬식
DDPM	강의 7	Forward/Reverse Process, 노이즈 스케줄, ϵ-예측
ODE/SDE 기초	강의 9 (또는 수학 배경)	상미분방정식, Euler method, 확률 미분방정식 기초
Score Matching	강의 8	Score function $\nabla_x \log p(x)$, Denoising Score Matching

> 💡 **왜 이 선수 지식들이 모두 필요한가?**
>
> Flow Matching은 기존 생성 모델링의 여러 갈래를 **하나의 통합된 프레임워크**로 묶는 이론이다. 각 선수 지식이 필요한 구체적 이유를 이해하면 강의 전체의 흐름이 보인다:
>
> - **Normalizing Flow**: Flow Matching은 Continuous Normalizing Flow(CNF)를 "제대로" 학습하는 방법이다. 이산 NF의 Change of Variables 공식이 연속

> 시간으로 어떻게 확장되는지 알아야, 왜 기존 CNF 학습이 비쌌는지를 이해할 수 있다.
> - **DDPM**: Flow Matching이 "왜 더 좋은가"를 판단하려면, 비교 대상인 DDPM의 작동 방식(특히 노이즈 스케줄 $\bar{\alpha}_t$, 경로 $x_t = \sqrt{\bar{\alpha}_t} x_0 + \sqrt{1 - \bar{\alpha}_t}\epsilon$)을 정확히 알아야 한다. 섹션 4에서 DDPM이 Flow Matching의 특수한 경우임을 수학적으로 보여줄 것이다.
> - **ODE/SDE 기초**: Flow Matching의 핵심은 ODE $dx/dt = v_\theta(x, t)$를 푸는 것이다. Euler method가 왜 $x_{t+h} = x_t + h \cdot v(x_t, t)$인지, 이 근사의 오차가 어떻게 발생하는지 직관적으로 알아야 한다. 또한 섹션 7에서 SDE 확장을 다루므로, dW_t (Wiener process)의 기본 개념이 필요하다.
> - **Score Matching**: 섹션 7에서 Flow Matching을 SDE로 확장할 때, score function $\nabla_x \log p_t(x)$가 등장한다. Denoising Score Matching의 "조건부로 학습하면 주변부가 학습된다"는 원리가 Flow Matching의 핵심 정리 (Conditional FM = Marginal FM)와 **동일한 수학적 구조**이므로, 이 유사성을 인지하면 이해가 깊어진다.

학습 목표

이 강의를 마치면 다음을 할 수 있다:

1. **Continuous Normalizing Flow (CNF)**의 개념과 기존 학습 방법의 한계를 설명할 수 있다.
2. **Conditional Flow Matching Loss**를 유도하고, 왜 시뮬레이션 없이 학습이 가능한지 증명의 핵심을 이해한다.
3. **직선 보간 경로**가 왜 최적인지 Optimal Transport 관점에서 설명할 수 있다.
4. DDPM과 Flow Matching의 **수학적 관계**(경로, 예측 대상, 손실 함수)를 정확히 기술할 수 있다.
5. **Rectified Flow**의 Reflow 절차와 1-step 생성 가능성을 이해한다.
6. PyTorch로 Flow Matching **학습 및 샘플링 코드**를 구현할 수 있다.
7. **OT-CFM**, **고차 ODE 솔버(Heun, RK4)**, **CFG** 등 실전 기법을 적용할 수 있다.

핵심 논문

> 📄 Lipman et al., "Flow Matching for Generative Modeling", ICLR 2023 📄 Albergo & Vanden-Eijnden, "Building Normalizing Flows with Stochastic Interpolants", ICLR 2023 📄 Liu et al., "Flow Straight and Fast: Learning to Generate and Transfer Data with Rectified Flow", ICLR 2023 📄 Tong et al., "Improving and Generalizing Flow-Based Generative Models with Minibatch Optimal Transport", TMLR 2024 📄 Esser et al., "Scaling Rectified Flow Transformers for High-Resolution Image Synthesis" (Stable Diffusion 3), ICML 2024
>
> 참고 강의: MIT 6.S184 (2025, 2026) 전체 커리큘럼의 핵심

패러다임 비교 요약

Flow Matching은 DDPM을 대체하는 최신 생성 패러다임이다. **Stable Diffusion 3, FLUX, Movie Gen** 등 최신 모델이 모두 Flow Matching 기반이다.

	DDPM (강의 7)	**Flow Matching**
수학적 기반	SDE (확률 미분방정식)	**ODE** (상미분방정식)
경로	구불구불 (확률적)	**직선에 가까움**
학습 목표	노이즈 ϵ 예측	**속도** v 예측
샘플링	1000 (DDPM), 50 (DDIM)	**10~50** (Euler)
이론	Score Matching, 마르코프 체인	**Continuous Normalizing Flow**
구현 난이도	노이즈 스케줄 설계 필요	**단순** (직선 보간)
노이즈 추가	샘플링 시 노이즈 추가 (SDE)	**노이즈 없음** (결정론적)

⚠️ **중요**: Flow Matching은 DDPM과 완전히 별개가 아니라, 가우시안 경로를 사용하면 수학적으로 동치가 된다 (섹션 4 참조). 핵심 차이는 **직선 경로 선택**에 있다.

---

1. Continuous Normalizing Flow (CNF) 복습

> 💡 **왜 CNF를 먼저 복습하는가?**
>
> Flow Matching을 이해하려면, 먼저 "Flow Matching이 **무엇의 문제를 해결**하는 것인지"를 정확히 알아야 한다. CNF는 이론적으로 매우 강력한 생성 모델이지만, 학습이 너무 비싸서 실용적이지 못했다. Flow Matching은 바로 이 CNF를 "시뮬레이션 없이 학습하는 방법"을 제시한 것이다. 따라서 CNF의 구조와 그 한계를 정확히 이해하는 것이 Flow Matching 전체를 관통하는 핵심이다.

1.1 강의 2의 Normalizing Flow와의 관계

강의 2에서 배운 Normalizing Flow:

$$z_0 \xrightarrow{f_1} z_1 \xrightarrow{f_2} z_2 \xrightarrow{f_3} \cdots \xrightarrow{f_K} x$$

각 f_k가 **가역**이어야 하고, **야코비안의 행렬식**을 계산해야 했다.

여기서 **야코비안 행렬식(Jacobian determinant)**이란, 변환 f_k가 공간을 얼마나 "늘리거나 줄이는지"를 측정하는 양이다. 확률 밀도가 변환 아래에서 어떻게 바뀌는지를 추적하기 위해 필요하며, 구체적으로 Change of Variables 공식은 다음과 같다:

$$p_X(x) = p_Z(f^{-1}(x)) \cdot \left| \det \frac{\partial f^{-1}(x)}{\partial x} \right|$$

이산 NF의 한계는 명확하다: (1) 각 레이어 f_k가 **가역**이어야 하므로 아키텍처 선택이 제한되고, (2) 야코비안 행렬식 계산이 일반적으로 $O(d^3)$이므로 (여기서 d는 데이터 차

원) 고차원 데이터에 비용이 크다. 이를 해결하기 위해 Coupling Layer (RealNVP)나 Autoregressive Flow 등 특수한 구조를 사용했지만, 표현력에 제약이 생긴다.

CNF는 이를 **연속 버전**으로 확장:

$$\frac{dx_t}{dt} = v_\theta(x_t, t), \quad t \in [0, 1]$$

- $x_0 \sim p_{\text{init}} = \mathcal{N}(0, I)$ (노이즈)
- $x_1 \sim p_{\text{data}}$ (데이터)
- v_θ: **속도장** (velocity field), 뉴럴넷으로 파라미터화

> 이산적 레이어 $f_1, f_2, \ldots$를 **무한히 많은, 무한히 얇은 레이어**로 바꾼 것이 CNF.

여기서 **속도장(velocity field)**이란, 데이터 공간의 각 점 x와 각 시간 t에서 "데이터가 어느 방향으로, 얼마나 빠르게 이동해야 하는가"를 알려주는 벡터 함수이다. 물리학에서 유체의 속도장과 동일한 개념으로, 각 입자(=데이터 포인트)가 이 속도장을 따라 흐르면(flow), 시간 $t = 0$의 노이즈 분포가 시간 $t = 1$의 데이터 분포로 변환된다. 이것이 "flow"라는 이름의 유래이다.

CNF의 핵심 장점은 이산 NF와 달리 **가역 아키텍처의 제약이 없다**는 것이다. ODE $dx_t/dt = v_\theta(x_t, t)$의 해는 v_θ가 Lipschitz 연속이면 자동으로 가역이 되기 때문이다 (Picard-Lindelof 정리). 즉 **임의의 뉴럴넷**을 v_θ로 사용할 수 있다.

⚠️ **표기법 주의**: Flow Matching에서는 $t = 0$이 노이즈, $t = 1$이 데이터인 경우가 많다. 이는 DDPM의 일반적 표기법($t = 0$이 데이터, $t = T$가 노이즈)과 **반대**이다. 논문마다 표기법이 다를 수 있으므로 항상 확인해야 한다.

1.2 CNF의 문제점 (기존)

> 💡 **왜 기존 CNF가 실용적이지 못했는가?**
>
> 아이디어는 아름답지만, 학습 과정에서의 계산 비용이 너무 컸다. 핵심 병목은 두 가지: (1) 학습의 매 스텝마다 ODE를 수치적으로 풀어야 하고, (2) log-likelihood를 계산하려면 속도장의 divergence를 추적해야 한다. 이 두 문제를 구체적으로 살펴보자.

기존 CNF (Neural ODE, Chen et al. 2018)는 학습에 **ODE를 시뮬레이션**해야 했다:

- 역전파를 위해 adjoint method 사용 → 느리고 메모리 많이 사용
- log-likelihood 계산에 trace estimator 필요 → 분산 큼

문제 1: ODE 시뮬레이션 비용 — Forward와 Backward 모두 비싸다

CNF의 학습 목표는 보통 Maximum Likelihood, 즉 $\max_\theta \mathbb{E}_{x_1 \sim p_{\text{data}}}[\log p_\theta(x_1)]$이다. 이를 계산하려면:

1. **Forward Pass**: 데이터 x_1에서 시작해 ODE를 **역방향으로** ($t = 1 \to t = 0$) 풀어 대응하는 잠재 변수 x_0을 구한다. 이 과정에서 ODE 솔버가 L번의 함수 평가

(function evaluation)를 수행한다.

2. **Change of Variables (연속 버전)**: 이산 NF의 야코비안 행렬식이 연속 시간에서는 다음과 같이 확장된다:

$$\log p_1(x_1) = \log p_0(x_0) - \int_0^1 \nabla \cdot v_\theta(x_t, t)\, dt$$

여기서 $\nabla \cdot v_\theta = \sum_{i=1}^d \frac{\partial v_{\theta,i}}{\partial x_i}$는 속도장의 **divergence**(발산)이다. 이 적분은 ODE를 풀면서 **동시에** 계산해야 하므로, 확장된 ODE 시스템을 풀게 된다.

3. **Backward Pass (Gradient 계산)**: $\log p_\theta(x_1)$에 대한 θ의 기울기를 구하려면, adjoint method로 ODE를 **다시 역방향으로** 풀어야 한다. 즉, forward에서 L번, backward에서 또 L번, 총 $O(L)$ **순차적 연산**이 필요하다.

결과적으로 **한 번의 학습 스텝**에 ODE 솔버를 두 번 (forward + backward) 돌려야 하고, 각각 수십~수백 번의 뉴럴넷 호출을 포함한다. 이것은 일반적인 딥러닝 학습(forward 1번 + backward 1번)에 비해 **수십~수백 배 느리다**.

문제 2: Divergence 계산 — $O(d)$ 비용 또는 높은 분산

위 Change of Variables 공식에서 divergence $\nabla \cdot v_\theta(x_t, t)$를 정확히 계산하려면, d차원 출력의 각 성분을 각 입력 성분으로 편미분해야 한다. 이는 d번의 역전파(backprop)를 의미하며, 이미지($d = 3 \times 256 \times 256 \approx 200,000$)에서는 사실상 불가능하다.

대안으로 **Hutchinson's trace estimator**를 사용한다:

$$\nabla \cdot v \approx \mathbb{E}_{\epsilon \sim \mathcal{N}(0,I)}[\epsilon^T \cdot \frac{\partial v}{\partial x} \cdot \epsilon]$$

이 추정량은 한 번의 역전파로 계산 가능하지만 (vector-Jacobian product 활용), **분산이 크다**. 특히 고차원에서 분산이 더 커지므로 학습이 불안정해진다.

📄 Chen et al., "Neural Ordinary Differential Equations", NeurIPS 2018 — Neural ODE의 원 논문

Flow Matching의 핵심 기여: 시뮬레이션 없이 CNF를 학습하는 방법을 찾은 것.

이제 핵심 질문은 이것이다: ODE를 풀지 않고, divergence를 계산하지 않고, 어떻게 CNF의 속도장 v_θ를 학습할 수 있는가? 이 질문에 대한 답이 바로 섹션 2의 Flow Matching이다.

---

2. Flow Matching: 핵심 아이디어

> 💡 **왜 Flow Matching이 혁명적인가?**
>
> 섹션 1에서 보았듯이, CNF는 학습할 때마다 ODE를 수치적으로 풀어야 했다 (simulation-based training). Flow Matching은 이 문제를 완전히 우회한다: **ODE를**

> **풀지 않고도** 속도장을 학습할 수 있는 simple regression loss를 제시한다. 이것이 가능한 이유는 "주변 속도장을 직접 학습할 필요 없이, 조건부 속도장을 학습하면 된다"는 핵심 정리 덕분이다. 이 섹션에서 이 정리의 의미와 증명을 깊이 이해하는 것이 강의 전체에서 가장 중요하다.

2.1 목표

속도장 $v_\theta(x,t)$를 학습하여, ODE $dx/dt = v_\theta(x,t)$를 $t=0$에서 $t=1$까지 시뮬레이션하면 $\mathcal{N}(0,I)$이 p_{data}로 변환되도록 한다.

좀 더 정확하게 말하면, 우리가 원하는 것은 다음 조건을 만족하는 속도장 v_θ를 찾는 것이다:

- 시간 t에서의 **확률 경로**(probability path) $p_t(x)$가 존재하여, $p_0 = \mathcal{N}(0,I)$이고 $p_1 = p_{\text{data}}$
- 이 확률 경로가 속도장 v_θ에 의해 생성된다. 즉, **연속 방정식**(continuity equation)을 만족한다:

$$\frac{\partial p_t(x)}{\partial t} + \nabla \cdot \left(p_t(x)\, v_\theta(x,t)\right) = 0$$

이 연속 방정식은 물리학에서 유체의 질량 보존 법칙과 동일하다: 확률(=질량)이 속도장을 따라 흐를 때, 어떤 영역에서 확률이 빠져나가면 그만큼 그 영역의 확률 밀도가 줄어든다. 이 방정식을 만족하는 v_θ를 찾으면, ODE를 풀어서 노이즈를 데이터로 변환할 수 있다.

2.2 이상적인 학습 목표 (하지만 사용 불가)

이상적으로는 **주변 속도장**(marginal velocity field) $u_t(x)$에 맞추고 싶다:

$$\mathcal{L}_{\text{FM}}(\theta) = \mathbb{E}_{t,x\sim p_t(x)}\left[\|v_\theta(x,t) - u_t(x)\|^2\right]$$

문제: $u_t(x)$와 $p_t(x)$를 모른다 (전체 데이터 분포에 의존).

여기서 **주변 속도장** $u_t(x)$가 무엇인지 정확히 이해하자. $u_t(x)$는 시간 t, 위치 x에서 "모든 가능한 데이터 포인트 x_1으로의 조건부 속도를 확률적으로 가중 평균한 것"이다:

$$u_t(x) = \frac{\int u_t(x \mid x_1)\, p_t(x \mid x_1)\, p_{\text{data}}(x_1)\, dx_1}{p_t(x)}$$

여기서 $p_t(x) = \int p_t(x \mid x_1)\, p_{\text{data}}(x_1)\, dx_1$은 **주변 확률 경로**이다. 이 식을 계산하려면 p_{data}에 대한 적분이 필요하고, p_{data} 자체를 모르므로 (그것이 우리가 학습하려는 대상이다!) 직접 계산은 불가능하다.

⚠️ **핵심 난점**: 주변 속도장 $u_t(x)$는 모든 데이터 포인트의 조건부 속도장을 주변화(marginalize)한 것이므로, 전체 데이터 분포를 알아야 계산할 수 있다. 이것이 바로 조건부(Conditional) 접근이 필요한 이유이다.

2.3 조건부 Flow Matching (Conditional FM) — 핵심 트릭

> 💡 **왜 직선 보간(linear interpolation)을 경로로 선택하는가?**
>
> 무한히 많은 경로 중에서 직선을 선택하는 것은 단순히 "쉬워서"가 아니다. 이것은 **Optimal Transport 이론에서 최적인 경로**이기 때문이다. 두 점 분포(point mass) δ_{x_0}와 δ_{x_1} 사이의 **Wasserstein-2 측지선(geodesic)**은 바로 직선 보간 $(1-t)x_0 + tx_1$이다. 이 경로는 **운동 에너지(kinetic energy)** $\int_0^1 \|dx_t/dt\|^2 dt$를 최소화한다. 직선 경로에서 속도는 $x_1 - x_0$으로 상수이므로, 운동 에너지는 $\|x_1 - x_0\|^2$이 되어, 가능한 모든 경로 중 가장 작다. 곡선 경로는 같은 거리를 가더라도 속도가 변하므로 운동 에너지가 더 크다 (Jensen 부등식에 의해).

개별 데이터 포인트 x_1에 대한 **조건부 경로**를 정의한다:

$$\psi_t(x_0 \mid x_1) = (1-t)\, x_0 + t\, x_1$$

- $t = 0$: $\psi_0 = x_0$ (노이즈)
- $t = 1$: $\psi_1 = x_1$ (데이터)
- 중간: **직선 보간** (linear interpolation)

이 경로의 조건부 속도장:

$$u_t(x \mid x_1) = \frac{d\psi_t}{dt} = x_1 - x_0$$

[단계별 유도] 이 결과를 단계별로 유도해 보자:

$$\psi_t = (1-t) \cdot x_0 + t \cdot x_1$$

시간 t에 대해 미분하면:

$$\frac{d\psi_t}{dt} = \frac{d}{dt}[(1-t) \cdot x_0 + t \cdot x_1] = \frac{d}{dt}[(1-t)] \cdot x_0 + \frac{d}{dt}[t] \cdot x_1 = -x_0 + x_1 = x_1 - x_0$$

여기서 x_0과 x_1은 고정된 상수(한 번 샘플링한 노이즈와 데이터)이므로 t에 의존하지 않는다. 따라서 속도 $u_t(x \mid x_1) = x_1 - x_0$은 **시간 t에 무관한 상수**이다.

> 속도는 t에 **무관**하고, 단순히 "노이즈에서 데이터로의 방향 벡터".

이 "상수 속도"라는 성질이 매우 중요하다. 속도가 시간에 따라 변하지 않으므로, ODE 솔버가 곡선을 추적할 필요가 없다. 이론적으로 **Euler method 1스텝**만으로도 정확한 해를 구할 수 있다 (개별 조건부 경로에 대해서). 실제로는 서로 다른 (x_0, x_1) 쌍의 속도장이 합쳐지면서 주변 속도장은 약간 비선형이 되지만, 여전히 매우 완만하므로 적은 스텝으로 충분하다.

2.4 Conditional Flow Matching Loss

$$\boxed{\mathcal{L}_{\text{CFM}}(\theta) = \mathbb{E}_{t\sim U[0,1],\, x_0\sim\mathcal{N}(0,I),\, x_1\sim p_{\text{data}}} \left[\|v_\theta(\psi_t, t) - (x_1 - x_0)\|^2\right]}$$

여기서 $\psi_t = (1-t)x_0 + tx_1$.

이 손실 함수의 의미를 분해해 보자:

1. **시간 t를 균일하게 샘플링**: $t \sim U[0,1]$ — 모든 시간대를 균등하게 학습
2. **노이즈와 데이터를 각각 독립적으로 샘플링**: $x_0 \sim \mathcal{N}(0, I)$, $x_1 \sim p_{\text{data}}$ — 랜덤 페어링
3. **직선 보간으로 중간 지점 계산**: $\psi_t = (1-t)x_0 + tx_1$ — 학습 입력
4. **목표 속도와의 MSE 최소화**: $\|v_\theta(\psi_t, t) - (x_1 - x_0)\|^2$ — 단순 회귀 문제!

핵심은 **ODE를 풀 필요가 전혀 없다**는 것이다. 중간 지점 ψ_t를 직접 계산할 수 있고, 목표 속도 $x_1 - x_0$도 알고 있다. 이것은 단순한 지도 학습(supervised learning) 회귀 문제와 동일하다.

> **DDPM의 Simple Loss와 비교**:
>
> DDPM: $\mathbb{E}[\|\epsilon - \epsilon_\theta(x_t, t)\|^2]$ → **노이즈** 예측
>
> Flow Matching: $\mathbb{E}[\|v_\theta(\psi_t, t) - (x_1 - x_0)\|^2]$ → **속도(방향)** 예측

이 두 손실 함수의 구조적 유사성에 주목하라. 둘 다 (1) 중간 지점을 만들고, (2) 그 지점에서 "어디로 가야 하는가"를 예측하는 회귀 문제이다. 차이는 DDPM은 "어디에서 왔는가(노이즈 ϵ)"를 예측하고, FM은 "어디로 가야 하는가(속도 $x_1 - x_0$)"를 예측한다는 점이다. 이 차이가 섹션 4에서 자세히 다루는 수학적 관계로 이어진다.

핵심 정리 (Lipman et al., 2023)

Conditional FM Loss의 기울기는 Marginal FM Loss의 기울기와 **동일**하다:

$$\nabla_\theta \mathcal{L}_{\text{CFM}} = \nabla_\theta \mathcal{L}_{\text{FM}}$$

이 정리가 **Flow Matching이 작동하는 모든 것의 기반**이다. 이 등식이 성립하지 않으면, 조건부 속도를 학습하는 것이 주변 속도를 학습하는 것과 무관할 수 있고, 그러면 전체 프레임워크가 무너진다.

[증명의 핵심 아이디어 — 전기대값의 법칙(Law of Total Expectation)]

증명을 직관적으로, 그리고 핵심 수학적 단계를 밟아 이해하자.

Step 1: Marginal FM Loss를 쓴다:

$$\mathcal{L}_{\text{FM}}(\theta) = \mathbb{E}_t \mathbb{E}_{x \sim p_t(x)} \left[\|v_\theta(x, t) - u_t(x)\|^2 \right]$$

Step 2: 이 손실의 기울기를 θ에 대해 취한다. MSE를 전개하면:

$$\|v_\theta - u_t\|^2 = \|v_\theta\|^2 - 2\langle v_\theta, u_t \rangle + \|u_t\|^2$$

θ에 대한 기울기에서 $\|u_t\|^2$ 항은 θ와 무관하므로 사라진다:

$$\nabla_\theta \mathcal{L}_{\text{FM}} = \mathbb{E}_{t,x\sim p_t}\left[\nabla_\theta \|v_\theta(x,t)\|^2 - 2\nabla_\theta \langle v_\theta(x,t), u_t(x)\rangle\right]$$

Step 3: 마찬가지로 Conditional FM Loss의 기울기를 계산한다:

$$\nabla_\theta \mathcal{L}_{\text{CFM}} = \mathbb{E}_{t,x_1,x_0}\left[\nabla_\theta \|v_\theta(\psi_t,t)\|^2 - 2\nabla_\theta \langle v_\theta(\psi_t,t), u_t(\psi_t \mid x_1)\rangle\right]$$

Step 4: 핵심은 **두 번째 항**이 같다는 것을 보이는 것이다. $p_t(x) = \int p_t(x \mid x_1)p_{\text{data}}(x_1)dx_1$이므로:

$$\mathbb{E}_{x\sim p_t}[f(x)\cdot u_t(x)] = \mathbb{E}_{x_1\sim p_{\text{data}}}\mathbb{E}_{x\sim p_t(\cdot|x_1)}[f(x)\cdot u_t(x \mid x_1)]$$

이 등식은 **전기대값의 법칙**(law of total expectation)을 속도장에 적용한 것이다. 주변 속도장 $u_t(x)$의 정의에 의해, $p_t(x)\cdot u_t(x) = \int u_t(x \mid x_1)p_t(x \mid x_1)p_{\text{data}}(x_1)dx_1$이 성립한다.

Step 5: 첫 번째 항 $\mathbb{E}[\nabla_\theta \|v_\theta\|^2]$도 동일하다. $x \sim p_t(x)$에서 x를 뽑는 것이나, (x_0, x_1)을 뽑고 $\psi_t = (1-t)x_0 + tx_1$을 계산하는 것이나, ψ_t의 주변 분포는 동일하게 p_t이기 때문이다.

따라서 $\nabla_\theta \mathcal{L}_{\text{CFM}} = \nabla_\theta \mathcal{L}_{\text{FM}}$이 성립한다. □

> **핵심 통찰**: 두 손실 함수 자체($\mathcal{L}_{\text{FM}}$과 $\mathcal{L}_{\text{CFM}}$)는 일반적으로 같지 **않다**. 하지만 그 **기울기**가 같다. 경사 하강법은 기울기만 사용하므로, 두 손실 중 어느 것을 최적화하든 동일한 최적 해에 도달한다. 이는 "상수 차이만 있는 두 함수를 최적화하는 것은 동일하다"는 원리이다.

⚠️ **직관적 설명**: 이 정리가 성립하는 이유를 직관적으로 이해하면 — 우리가 조건부 경로들의 속도를 예측하도록 학습하면, 그 경로들을 모두 "평균"한 주변 속도장이 자연스럽게 학습된다. 이는 Denoising Score Matching에서 개별 노이즈 레벨에서의 학습이 전체 Score를 학습하는 것과 동일한 원리이다.

> 💡 **Denoising Score Matching과의 깊은 유사성**
>
> Denoising Score Matching(DSM)에서도 동일한 수학적 구조가 나타난다:
>
> - DSM: $\mathbb{E}_{x_0,\epsilon}[\|s_\theta(x_0+\sigma\epsilon) - (-\epsilon/\sigma)\|^2]$ — 노이즈가 추가된 개별 샘플에서 score를 학습
> - CFM: $\mathbb{E}_{x_0,x_1}[\|v_\theta(\psi_t,t) - (x_1 - x_0)\|^2]$ — 개별 (x_0, x_1) 쌍에서 속도를 학습
>
> 두 경우 모두 "전체 분포에 대한 양(marginal score / marginal velocity)"을 직접 계산하지 않고, "개별 샘플에 대한 조건부 양(conditional score / conditional velocity)"을 학습하면, 전기대값의 법칙에 의해 주변부가 자동으로 학습된다. Flow Matching은 이 아이디어를 score에서 velocity로 확장한 것이라고 볼 수 있다.

---

3. 학습 & 생성 알고리즘

> 💡 **왜 이 알고리즘이 놀라운가?**
>
> 기존 CNF 학습은 매 스텝마다 ODE 솔버를 돌려야 했다 (수십~수백 번의 뉴럴넷 호출). Flow Matching 학습은 **뉴럴넷을 딱 1번 호출**하고 MSE를 계산하면 끝이다. DDPM 학습과 동일한 수준의 단순함이면서, 이론적 기반은 더 깔끔하다.

3.1 학습

```
Algorithm: Flow Matching Training
━━━
```

```
Algorithm: Flow Matching Sampling
━━━
```

$$p_t(x \mid x_1) = \mathcal{N}(x;\, \sqrt{\bar{\alpha}_t}\, x_1,\, (1-\bar{\alpha}_t)I)$$

표기법을 Flow Matching과 통일하면 ($t=0$: 데이터, $t=1$: 노이즈로 바꾸면):

$$\psi_t^{\text{diff}}(x_0 \mid x_1) = \alpha_t\, x_1 + \sigma_t\, x_0$$

여기서 α_t, σ_t는 노이즈 스케줄에 의해 결정.

Flow Matching의 경로 (직선):

$$\psi_t^{\text{FM}}(x_0 \mid x_1) = (1-t)\, x_0 + t\, x_1$$

```
Flow Matching:          DDPM (Gaussian path):
x₀ ──────────── x₁      x₀ ─┐     ┌── x₁
  (직선)                      └─┘
                        (가우시안 곡선)
```

이 두 경로의 차이를 구체적으로 이해하자:

DDPM의 경로 $\psi_t = \alpha_t x_1 + \sigma_t x_0$에서 α_t와 σ_t는 비선형 함수이다. 예를 들어 cosine schedule에서는 $\alpha_t = \cos(\pi t/2)$, $\sigma_t = \sin(\pi t/2)$이다. 이것을 데이터 공간에서 시각화하면, ψ_t는 x_0에서 x_1로 **곡선을 그리며** 이동한다. 구체적으로:

- $t=0$: $\psi_0 = 0 \cdot x_1 + 1 \cdot x_0 = x_0$ (노이즈)
- $t=0.5$ (cosine schedule): $\psi_{0.5} = \cos(\pi/4)x_1 + \sin(\pi/4)x_0 = \frac{\sqrt{2}}{2}(x_1 + x_0)$ — 이것은 직선 보간의 $0.5x_0 + 0.5x_1$과 다르다!
- $t=1$: $\psi_1 = 1 \cdot x_1 + 0 \cdot x_0 = x_1$ (데이터)

FM의 경로 $\psi_t = (1-t)x_0 + tx_1$은 말 그대로 x_0에서 x_1로의 직선이다. t에 대해 선형이므로, 데이터 공간에서 최단 거리를 따라 이동한다.

왜 직선 경로가 ODE 솔버에 유리한가? DDPM의 곡선 경로에서는 속도 $u_t = \alpha_t' x_1 + \sigma_t' x_0$이 시간 t에 따라 **크게 변한다** (α_t'와 σ_t'가 비선형이므로). 이는 ODE 솔버가 속도 변화를 정확히 추적해야 함을 의미하며, 많은 스텝이 필요하다. 반면 FM의 직선 경로에서 속도 $u_t = x_1 - x_0$은 **상수**이므로, Euler method조차 (주변 속도장 수준에서) 매우 정확하다.

4.2 Gaussian Flow Matching ≈ Diffusion

> 💡 **왜 이 동치가 중요한가?**
>
> 이 동치는 DDPM이 "특별한 노이즈 스케줄을 선택한 Flow Matching"에 불과하다는 것을 보여준다. 즉, **경로의 선택만 바꾸면** DDPM에서 FM으로 전환할 수 있다. 기존 DDPM 아키텍처(U-Net, attention 등)를 그대로 사용하면서, 학습 목표와 경로만 바꾸면 FM의 장점을 얻을 수 있다.

가우시안 경로를 사용한 Flow Matching:

$$\psi_t(x_0 \mid x_1) = \alpha_t\, x_1 + \sigma_t\, x_0, \quad \alpha_0 = 0,\, \alpha_1 = 1,\, \sigma_0 = 1,\, \sigma_1 = 0$$

이 경우의 속도:

$$u_t = \frac{\alpha_t'}{\alpha_t}(\psi_t - \sigma_t x_0) + \sigma_t' x_0 = \alpha_t' x_1 + \sigma_t' x_0$$

[단계별 유도] 이 속도를 유도해 보자:

$$\psi_t = \alpha_t x_1 + \sigma_t x_0$$

시간 t에 대해 미분하면:

$$u_t = \frac{d\psi_t}{dt} = \alpha_t' x_1 + \sigma_t' x_0$$

여기서 $\alpha_t' = d\alpha_t/dt$, $\sigma_t' = d\sigma_t/dt$이다. FM의 직선 경로($\alpha_t = t$, $\sigma_t = 1 - t$)에서는 $\alpha_t' = 1$, $\sigma_t' = -1$이므로 $u_t = x_1 - x_0$이 된다. 즉 직선 경로는 $\alpha_t = t$인 특수한 경우이다.

그리고 DDPM의 스케줄 $\alpha_t = \sqrt{\bar{\alpha}_t}$, $\sigma_t = \sqrt{1 - \bar{\alpha}_t}$를 대입하면 **DDPM과 수학적으로 동치**가 된다.

[동치의 구체적 의미] DDPM의 reverse process에서의 ODE (DDIM ODE라고도 부른다)는:

$$\frac{dx_t}{dt} = \frac{\alpha_t'}{\alpha_t} x_t - \alpha_t' \sigma_t \left(\frac{1}{\alpha_t} + \frac{\sigma_t'}{\alpha_t' \sigma_t} \right) \cdot \underbrace{\frac{x_t - \alpha_t \hat{x}_1(x_t, t)}{\sigma_t}}_{\text{predicted noise } \hat{\epsilon}}$$

이것은 가우시안 경로 FM의 ODE와 정확히 동일한 형태이다. 차이는 경로(즉 α_t, σ_t의 선택)뿐이다. DDPM은 구불구불한 $\alpha_t = \sqrt{\bar{\alpha}_t}$를, FM은 직선인 $\alpha_t = t$를 사용한다.

4.3 세 가지 예측 대상의 변환

> 💡 **왜 세 가지 예측 대상을 이해해야 하는가?**
>
> 실무에서는 코드마다 다른 예측 대상을 사용한다. DDPM 코드는 ϵ을, Flow Matching 코드는 v를, 일부 코드는 x_1 (또는 x_0)을 예측한다. 하지만 이 셋은 수학적으로 **완전히 등가**이며, 하나를 알면 나머지 둘을 즉시 계산할 수 있다. 이 변환을 이해하면 어떤 코드든 읽을 수 있고, 필요에 따라 예측 대상을 전환할 수 있다.

$$\boxed{v_t = \alpha_t'\, x_1 + \sigma_t'\, x_0 = \alpha_t' \left(\frac{\psi_t - \sigma_t \epsilon}{\alpha_t} \right) + \sigma_t' \epsilon = \frac{\alpha_t'}{\alpha_t} \psi_t + \left(\sigma_t' - \frac{\alpha_t' \sigma_t}{\alpha_t} \right) \epsilon}$$

[단계별 유도] 이 변환을 단계별로 따라가자:

Step 1: $\psi_t = \alpha_t x_1 + \sigma_t x_0$이고, $x_0 = \epsilon$ (표준 가우시안 노이즈)이므로:

$$x_1 = \frac{\psi_t - \sigma_t \epsilon}{\alpha_t}$$

Step 2: 속도 $v_t = \alpha_t' x_1 + \sigma_t' x_0$에 위의 x_1을 대입:

$$\begin{aligned} v_t &= \alpha_t' \cdot \frac{\psi_t - \sigma_t \epsilon}{\alpha_t} + \sigma_t' \cdot \epsilon \\ &= \frac{\alpha_t'}{\alpha_t}\psi_t - \frac{\alpha_t' \sigma_t}{\alpha_t}\epsilon + \sigma_t' \epsilon \\ &= \frac{\alpha_t'}{\alpha_t}\psi_t + \left(\sigma_t' - \frac{\alpha_t' \sigma_t}{\alpha_t}\right)\epsilon \end{aligned}$$

Step 3: 따라서 v_t를 알면 ϵ을 역으로 구할 수 있고, ϵ을 알면 x_1을 구할 수 있다.

직선 경로($\alpha_t = t$, $\sigma_t = 1 - t$)에서의 특수한 경우:

$$v_t = \frac{1}{t}\psi_t + \left(-1 - \frac{1-t}{t}\right)\epsilon = \frac{1}{t}\psi_t - \frac{1}{t}\epsilon = \frac{\psi_t - \epsilon}{t}$$

이는 $v_t = x_1 - x_0 = x_1 - \epsilon$이므로, $x_1 = v_t + \epsilon = v_t + x_0$이다. 매우 직관적이다.

예측	기호	다른 예측으로의 변환
노이즈	ϵ	$v = \alpha_t' x_1 + \sigma_t' x_0$ 관계에서 유도
속도	v	$\epsilon = (\psi_t - \alpha_t x_1)/\sigma_t$
원본	x_1	$x_1 = (\psi_t - \sigma_t \epsilon)/\alpha_t$

실무에서의 선택 가이드: v-예측은 $t \approx 0$과 $t \approx 1$ 근방에서 수치적으로 안정적이다. 반면 ϵ-예측은 $t \approx 1$ ($\alpha_t \to 1$, $\sigma_t \to 0$)에서 $\epsilon = (\psi_t - \alpha_t x_1)/\sigma_t$의 분모가 0에 가까워져 불안정해질 수 있다. 이것이 최신 모델들(SD3, FLUX)이 v-예측을 선호하는 이유 중 하나이다.

---

5. 왜 Flow Matching이 더 좋은가?

5.1 직선 경로의 장점

> 💡 **왜 경로의 "직선성"이 생성 품질과 속도에 직접 영향을 미치는가?**
>
> ODE 기반 생성에서 경로의 곡률(curvature)은 ODE 솔버의 정확도를 직접 결정한다. 곡선 경로에서는 속도장이 급격히 변하므로, 솔버가 이 변화를 놓치지 않으려면 작은 스텝 크기(=많은 스텝)가 필요하다. 직선 경로에서는 속도장이 거의 상수이므로, 큰 스텝으로도 정확한 결과를 얻는다. 이것이 FM이 10-50 스텝으로 DDPM의 1000 스텝과 유사하거나 더 나은 결과를 내는 이유이다.

OT(Optimal Transport) 관점에서, 직선 보간 경로는 **가장 효율적인 이동 경로**이다:

장점	설명
빠른 수렴	직선이므로 Euler method의 오차가 작음
적은 스텝	10~50 스텝으로 충분 (DDPM: 1000)
단순한 학습	목표 = $x_1 - x_0$ (상수 속도)
안정적 학습	경로 교차가 적어 모드 붕괴 위험 낮음

⚠️ **Optimal Transport 관점의 보충 설명**: 두 분포 p_0과 p_1 사이의 OT 경로는 Wasserstein-2 거리를 최소화하는 경로이며, 이 최적 경로가 바로 직선 보간이다. 즉 Flow Matching의 직선 경로는 단순한 선택이 아니라, 이론적으로 최적인 선택이다.

Wasserstein-2 거리와 직선 경로의 관계를 구체적으로 이해하자:

Wasserstein-2 거리는 두 분포 사이에 "질량을 운반하는 최소 비용"을 측정한다:

$$W_2(p_0, p_1) = \left(\inf_{\gamma \in \Pi(p_0, p_1)} \int \|x_0 - x_1\|^2 \, d\gamma(x_0, x_1) \right)^{1/2}$$

여기서 $\Pi(p_0, p_1)$는 p_0과 p_1을 주변 분포(marginal)로 갖는 모든 결합 분포(coupling)의 집합이다.

이 최적 운반 계획(optimal transport plan) γ^*가 주어지면, 두 분포를 연결하는 **Wasserstein 측지선(geodesic)**은:

$$p_t = [(1-t) \cdot \text{proj}_0 + t \cdot \text{proj}_1]_{\#} \gamma^*$$

즉, 최적 결합에서 뽑은 (x_0, x_1) 쌍을 직선 $(1-t)x_0 + tx_1$으로 연결하는 것이다. 이 측지선은 가능한 모든 확률 경로 중에서 **총 운동 에너지** $\int_0^1 \mathbb{E}[\|dx_t/dt\|^2] dt$를 최소화한다.

Flow Matching의 직선 보간은 바로 이 Wasserstein 측지선의 **조건부 버전**이다. 각 (x_0, x_1) 쌍을 직선으로 연결하므로, 개별 경로 수준에서 최적이다. 다만 (x_0, x_1)의 페어링이 랜덤이므로 전역적(global) OT와는 다르다 — 이 차이를 개선하는 것이 섹션 9의 OT-CFM이다.

5.2 벤치마크 비교

ImageNet 256x256 FID:

모델	방법	샘플링 스텝	FID↓
DDPM	SDE	1000	4.9
DDIM	ODE	50	4.7
OT-CFM	**Flow Matching**	**50**	**~3.5**

DiT (DDPM 학습)	SDE/ODE	250	2.27
DiT (FM 학습)	ODE	50	~2.1

주목할 점은 **동일한 아키텍처(DiT)**에서 학습 방법만 DDPM에서 FM으로 바꾸었을 때, 더 적은 샘플링 스텝으로 더 나은 FID를 달성한다는 것이다. 이는 FM의 장점이 아키텍처가 아닌 **경로 선택과 학습 목표**에서 온다는 것을 확인시켜 준다.

5.3 SD 3에서의 실제 사용

📄 Esser et al., "Scaling Rectified Flow Transformers for High-Resolution Image Synthesis", ICML 2024

Stable Diffusion 3는 Flow Matching을 사용한다:

```
z_T ~ N(0, I)
for t in [0, 1] with N steps:
    v = MM-DiT(z_t, t, c_text)      # 속도 예측
    z_{t+h} = z_t + h · v            # Euler step (노이즈 없음!)
z_1 → [VAE Decoder] → 이미지
```

⚠️ **SD3의 세부 사항**: Stable Diffusion 3는 단순한 Flow Matching이 아닌, **Rectified Flow** + **MM-DiT (Multi-Modal Diffusion Transformer)** 아키텍처를 사용한다. 또한 logit-normal 시간 샘플링을 도입하여 중간 시간대($t \approx 0.5$)에 더 많은 학습 리소스를 배분한다.

logit-normal 시간 샘플링에 대한 보충 설명: 표준 FM은 $t \sim U[0,1]$을 사용하지만, SD3는 $t = \sigma(\text{logit-normal})$을 사용한다. 이는 $t \approx 0.5$ 근처에 더 많은 샘플을 배치하고, $t \approx 0$과 $t \approx 1$ 근처에는 적은 샘플을 배치한다. 직관적 이유: $t \approx 0$ (거의 순수 노이즈)이나 $t \approx 1$ (거의 깨끗한 데이터)에서는 학습할 것이 적고, 중간 영역에서 속도장이 가장 복잡하므로 거기에 학습 리소스를 집중하는 것이 효율적이다.

---

6. Rectified Flow — 경로를 더 직선으로

> 💡 **왜 경로를 "더" 직선으로 만들어야 하는가?**
>
> 섹션 2에서 보았듯이, Flow Matching의 개별 조건부 경로 $(1-t)x_0 + tx_1$은 직선이다. 하지만 문제가 있다: x_0와 x_1이 **랜덤하게 페어링**되므로, 서로 다른 쌍의 경로가 **교차(cross)**할 수 있다. 경로가 교차하면, 같은 위치 x에서 시간 t에 서로 다른 방향의 속도가 요구된다. 네트워크는 이 모순된 요구를 평균내어 처리하므로, 결과적으로 주변 속도장이 비선형이 되고 ODE 솔버에 더 많은 스텝이 필요해진다. Rectified Flow는 이 교차를 줄여서, 주변 속도장 자체를 더 직선에 가깝게 만든다.

6.1 아이디어

> 📄 Liu et al., "Flow Straight and Fast: Learning to Generate and Transfer Data with Rectified Flow", ICLR 2023

Flow Matching의 경로가 개별적으로는 직선이지만, 다른 데이터 포인트들의 경로와 **교차**할 수 있다. Rectified Flow는 이 교차를 줄인다.

경로 교차의 구체적 예시: 2D에서 두 쌍을 생각해 보자:

- 쌍 1: $x_0^{(1)} = (-1, 0) \to x_1^{(1)} = (1, 0)$ (왼쪽에서 오른쪽으로)
- 쌍 2: $x_0^{(2)} = (0, -1) \to x_1^{(2)} = (0, 1)$ (아래에서 위로)

이 두 직선 경로는 원점 $(0, 0)$ 근처에서 교차한다. 원점에서 네트워크는 "오른쪽으로 가라"와 "위로 가라"는 모순된 신호를 받고, 그 평균인 "오른쪽 위로 가라"를 학습한다. 이 평균화된 속도장은 어느 경로도 정확히 따르지 못하므로, ODE 솔버가 더 많은 스텝을 써야 올바른 목적지에 도달한다.

6.2 Reflow 절차

```
1단계: 표준 Flow Matching 학습 → 모델 v₁

2단계 (Reflow):
    x₀ ~ N(0, I)
    x₁ = ODE_solve(v₁, x₀, t=0→1)   # 1단계 모델로 생성한 쌍
    이 (x₀, x₁) 쌍으로 새로운 FM 학습 → 모델 v₂
```

Reflow를 반복할수록 경로가 더 직선에 가까워지고, 1-step 생성도 가능해진다.

[Reflow가 작동하는 수학적 이유]

핵심 통찰: 1단계 모델 v_1의 ODE를 풀어 얻은 (x_0, x_1) 쌍은 더 이상 **랜덤 페어링이 아니다**. 이 쌍은 학습된 ODE flow에 의해 **실제로 연결되는** 쌍이다. 따라서:

1. **1단계 FM**: (x_0, x_1)이 독립적으로 샘플링됨 → 경로 교차 많음 → 주변 속도장이 비선형
2. **Reflow 후**: (x_0, x_1)이 ODE flow로 연결됨 → 경로 교차가 줄어듦 → 주변 속도장이 더 선형에 가까움
3. **2-3번 반복 후**: 경로가 거의 직선이 됨 → **1-step 생성이 가능**

수학적으로, Reflow는 **OT(Optimal Transport) 결합(coupling)**에 점점 가까워진다. OT 이론에 따르면, Wasserstein-2 최적 운반 계획에서의 경로는 교차하지 않는다 (이를 "monotonicity" 또는 "non-crossing" 성질이라 한다). Reflow의 각 반복은 이 최적 결합에 더 가까운 (x_0, x_1) 페어링을 만들어내므로, 경로 교차가 점진적으로 줄어든다.

1-step 생성이 가능한 이유: Reflow를 충분히 반복하면, 주변 속도장 $u_t(x)$가 거의 t에 무관한 상수가 된다. 즉, 시간 $t = 0$에서의 속도 $u_0(x_0)$가 전체 경로를 결정한다. 이 경

우 Euler method 1스텝: $x_1 = x_0 + 1 \cdot v_\theta(x_0, 0)$이 거의 정확한 결과를 준다. 이것이 **distillation 없이** 1-step 생성을 가능하게 하는 원리이다.

⚠️ **직관적 이해**: 초기 Flow Matching에서는 x_0와 x_1이 랜덤하게 페어링되므로, 서로 다른 (x_0, x_1) 쌍의 직선 경로가 교차할 수 있다. Reflow는 학습된 ODE를 통해 "실제로 연결되는" (x_0, x_1) 쌍을 찾아 재학습하므로, 경로 교차가 줄어든다.

⚠️ **실무적 고려사항**: Reflow는 추가 학습 라운드가 필요하고, 각 라운드에서 ODE를 풀어 (x_0, x_1) 쌍을 생성해야 하므로 비용이 든다. 실무에서는 보통 1-2회의 Reflow만 수행하며, SD3에서는 Rectified Flow를 1회 적용한 결과를 사용한다.

---

7. SDE 확장: Flow Matching + Diffusion

> 💡 **왜 결정론적인 ODE에 굳이 노이즈(확률적 요소)를 추가하는가?**
>
> ODE 기반 생성(Flow Matching)은 결정론적이므로, 동일한 초기 노이즈에서 항상 같은 결과가 나온다. 이것은 장점이기도 하지만, **모드 커버리지(mode coverage)** 측면에서 단점이 될 수 있다. ODE의 경로가 데이터 분포의 일부 모드를 놓칠 수 있기 때문이다. SDE로 확장하면, 샘플링 과정에서 추가된 노이즈가 경로를 약간씩 "흔들어" 더 다양한 영역을 탐색하게 해 준다. 이는 DDPM이 SDE를 사용하여 좋은 모드 커버리지를 달성했던 것과 같은 원리이다. FM의 SDE 확장은 "ODE의 효율성"과 "SDE의 다양성" 사이의 트레이드오프를 조절할 수 있게 해 준다.

7.1 MIT 6.S184의 통합 프레임워크

Flow Matching(ODE)을 SDE로 확장:

$$dX_t = v_\theta(X_t, t)\, dt + \sigma_t\, dW_t$$

여기서 dW_t는 **Wiener process(브라운 운동)**의 미소 증분이다. 직관적으로, dW_t는 각 시간 스텝에서 추가되는 "랜덤 섭동"이다. $dW_t \sim \mathcal{N}(0, dt \cdot I)$이므로, 작은 시간 구간 dt 동안 표준편차 $\sqrt{dt}$ 크기의 랜덤 노이즈가 더해진다.

- $\sigma_t = 0$: Flow Matching (ODE, 결정론적)
- $\sigma_t > 0$: Diffusion 추가 (SDE, 확률적)

σ_t의 크기를 조절하여 결정론적 ODE와 확률적 SDE 사이의 연속적인 스펙트럼을 탐색할 수 있다. 실무에서는 작은 σ_t를 사용하여 ODE의 효율성을 유지하면서 약간의 다양성을 추가하는 전략이 효과적이다.

학습 목표에 **스코어 항**이 추가:

$$\text{SDE 목표} = \mathbb{E}\left[\|v_\theta(\psi_t, t) - u_t\|^2\right] + \mathbb{E}\left[\|s_\theta(\psi_t, t) - \nabla \log p_t(\psi_t \mid x_1)\|^2\right]$$

첫 번째 항은 기존 FM의 속도 매칭이고, 두 번째 항은 **스코어 매칭**이다. 스코어 함수 $\nabla \log p_t(x)$는 "확률 밀도가 가장 빠르게 증가하는 방향"을 가리킨다. SDE에서 노이즈

가 입자를 잘못된 방향으로 밀어낼 때, 스코어 함수가 "확률 밀도가 높은 쪽으로 돌아오라"는 복원력을 제공한다.

7.2 가우시안 조건부 경로의 스코어

$$\nabla \log p_t(x \mid x_1) = -\frac{x - \alpha_t x_1}{\sigma_t^2} = -\frac{x_0}{\sigma_t}$$

[단계별 유도] 이 결과를 유도해 보자:

가우시안 조건부 경로에서 $p_t(x \mid x_1) = \mathcal{N}(x; \alpha_t x_1, \sigma_t^2 I)$이다. 가우시안의 log-density는:

$$\log p_t(x \mid x_1) = -\frac{\|x - \alpha_t x_1\|^2}{2\sigma_t^2} + \text{const}$$

x에 대해 기울기를 취하면:

$$\nabla_x \log p_t(x \mid x_1) = -\frac{x - \alpha_t x_1}{\sigma_t^2}$$

그리고 $x = \psi_t = \alpha_t x_1 + \sigma_t x_0$을 대입하면 $x - \alpha_t x_1 = \sigma_t x_0$이므로:

$$\nabla_x \log p_t(\psi_t \mid x_1) = -\frac{\sigma_t x_0}{\sigma_t^2} = -\frac{x_0}{\sigma_t}$$

> DDPM의 $-\epsilon/\sqrt{1-\bar{\alpha}_t}$와 동일한 형태.

이 결과는 "스코어 = 노이즈의 반대 방향 / 노이즈 스케일"이라는 직관적 의미를 가진다. 데이터에 노이즈 $\sigma_t x_0$가 추가되었으므로, 스코어는 그 노이즈를 제거하는 방향($-x_0/\sigma_t$)을 가리킨다.

7.3 Fokker-Planck 연결

> 💡 **왜 Fokker-Planck 방정식이 중요한가?**
>
> Fokker-Planck 방정식은 SDE 아래에서 **확률 밀도 함수가 시간에 따라 어떻게 변하는지**를 기술하는 편미분 방정식이다. 이것이 중요한 이유는: (1) ODE와 SDE가 동일한 확률 경로 p_t를 생성하는 조건을 찾을 수 있고, (2) Flow Matching의 연속 방정식이 Fokker-Planck의 특수한 경우임을 보여주어, 두 프레임워크의 이론적 통합을 가능하게 한다.

MIT 6.S184 Theorem 13에서:

ODE의 연속 방정식:

$$\frac{\partial p_t}{\partial t} + \nabla \cdot (p_t v_t) = 0$$

이 연속 방정식의 물리적 의미: 확률 "유체"가 속도장 v_t를 따라 흐를 때, 확률의 총량이 보존된다. $\nabla \cdot (p_t v_t)$는 각 점에서 확률의 "유출량"(flux의 divergence)이며, 이것이 $-\partial p_t / \partial t$ (밀도 감소율)와 같다.

SDE의 Fokker-Planck 방정식:

$$\frac{\partial p_t}{\partial t} = -\nabla \cdot (v_t p_t) + \frac{\sigma_t^2}{2} \nabla^2 p_t$$

연속 방정식과 비교하면, Fokker-Planck에는 **확산 항(diffusion term)** $\frac{\sigma_t^2}{2} \nabla^2 p_t$가 추가되었다. 이 항은 Laplacian($\nabla^2$)이므로, 확률 밀도를 "부드럽게" (smooth out) 만드는 효과가 있다. 물리학에서 열 방정식 $\partial T / \partial t = k \nabla^2 T$과 동일한 구조이다: 온도(=확률밀도)의 높은 곳에서 낮은 곳으로 열(=확률)이 퍼져나간다.

$\sigma_t = 0$이면 Fokker-Planck → 연속 방정식 (Flow Matching의 이론적 기반).

ODE-SDE 등가성: 흥미로운 사실은, 동일한 확률 경로 p_t를 생성하는 ODE와 SDE가 항상 존재한다는 것이다. 구체적으로, SDE $dX_t = v_t dt + \sigma_t dW_t$와 동일한 p_t를 생성하는 ODE는:

$$\frac{dx_t}{dt} = v_t(x_t, t) - \frac{\sigma_t^2}{2} \nabla \log p_t(x_t)$$

이것이 바로 Score SDE (Song et al., 2021)에서의 "probability flow ODE"이다. 이 등가성 덕분에, SDE로 학습한 모델을 ODE로 샘플링하거나, 그 반대로 하는 것이 가능하다.

---

8. Guidance in Flow Matching

> 💡 **왜 Guidance가 필요한가?**
>
> 조건부 생성(예: "고양이 이미지 생성")에서, 모델이 학습한 $p(x \mid c)$를 그대로 사용하면 다양하지만 조건과의 일치도가 낮은 샘플이 나올 수 있다. Classifier-Free Guidance(CFG)는 조건부 분포를 "강화"하여, 조건과 더 잘 일치하는 (하지만 덜 다양한) 샘플을 생성한다. 수학적으로 CFG는 $p(x \mid c)^w / Z$ (temperature scaling의 일종)에서 샘플링하는 것에 해당하며, $w > 1$일 때 조건 c에 더 집중된 분포에서 샘플링한다.

8.1 CFG in Flow Matching

DDPM에서의 CFG와 동일한 원리, 속도장에 적용:

$$\tilde{v}_\theta(x_t, t, c) = v_\theta(x_t, t, \varnothing) + w \cdot (v_\theta(x_t, t, c) - v_\theta(x_t, t, \varnothing))$$

이 식을 풀어 쓰면 $\tilde{v} = (1 - w) \cdot v_\theta(x_t, t, \varnothing) + w \cdot v_\theta(x_t, t, c)$이다. $w = 1$이면 순수한 조건부 속도, $w > 1$이면 "조건부 방향을 더 강하게 밀어주는" 효과가 있다.

$v_\theta(x_t, t, c) - v_\theta(x_t, t, \varnothing)$는 "조건 c가 속도장에 미치는 영향"을 의미하며, w는 이 영향의 강도를 조절한다.

학습 시 조건 드롭아웃도 동일하게 적용: 학습의 일정 비율(보통 10-20%)에서 조건 c를 null 토큰 $\varnothing$으로 대체하여, 동일한 모델이 조건부와 비조건부 속도장을 **동시에** 학습한다. 이렇게 하면 샘플링 시 추가 모델 없이 CFG를 적용할 수 있다.

8.2 Guidance Distillation (FLUX)

> 💡 **왜 Guidance Distillation이 필요한가?**
>
> CFG의 가장 큰 단점은 **모델을 2번 호출**해야 한다는 것이다 (조건부 1번 + 비조건부 1번). 이는 추론 비용을 정확히 2배로 만든다. 50스텝 생성이면 100번의 뉴럴넷 호출이 필요하다. Guidance Distillation은 이 2배 비용을 제거하는 기법이다.

FLUX에서는 가이던스를 **모델에 내장**:

1. CFG로 생성한 고품질 데이터 생성
2. 이 데이터로 가이던스 없는 모델을 재학습 (distillation)
3. 생성 시 모델을 **1번만** 호출 (2번이 아닌!)

→ 추론 속도 2배 향상

작동 원리: teacher 모델이 CFG($w > 1$)로 생성한 "고품질" 궤적(trajectory)을 student 모델이 CFG 없이($w = 1$) 재현하도록 학습한다. 즉, student는 teacher의 "가이드된 행동"을 내재화한다. 이는 knowledge distillation의 일종으로, teacher의 지식이 student의 파라미터에 녹아든다.

---

9. Optimal Transport Conditional Flow Matching (OT-CFM)

> 📄 Tong et al., "Improving and Generalizing Flow-Based Generative Models with Minibatch Optimal Transport", TMLR 2024

> 💡 **왜 OT-CFM이 필요한가?**
>
> 섹션 5에서 직선 경로가 Optimal Transport 측지선임을 보았다. 하지만 이것은 **개별 (x_0, x_1) 쌍에 대한** 최적성이지, **전체 분포에 대한** 최적성이 아니다. 표준 FM에서 x_0와 x_1이 랜덤하게 페어링되면, 먼 거리의 쌍이 연결되고 경로가 교차한다. OT-CFM은 미니배치 내에서 OT를 계산하여 "어떤 노이즈를 어떤 데이터에 연결할 것인가"를 최적화한다. 이는 Rectified Flow(섹션 6)와 유사한 목표를 추구하지만, 학습 중에 실시간으로 적용할 수 있다는 장점이 있다.

9.1 Mini-batch OT의 핵심 아이디어

표준 Flow Matching에서는 x_0와 x_1을 **독립적으로** 샘플링한다. 즉, 각 노이즈 x_0가 어떤 데이터 x_1과 페어링되는지는 **완전히 랜덤**이다.

OT-CFM은 미니배치 내에서 **Optimal Transport 매핑**을 먼저 계산하여, 가장 "가까운" (x_0, x_1) 쌍을 연결한다:

```
표준 FM:  x₀⁽¹⁾ ──────── x₁⁽³⁾   (먼 거리의 랜덤 페어링)
          x₀⁽²⁾ ──────── x₁⁽¹⁾   (경로 교차 발생)
          x₀⁽³⁾ ──────── x₁⁽²⁾

OT-CFM:   x₀⁽¹⁾ — x₁⁽¹⁾         (최적 매핑으로 짧은 거리)
          x₀⁽²⁾ — x₁⁽²⁾         (경로 교차 최소화)
          x₀⁽³⁾ — x₁⁽³⁾
```

[비용 행렬과 최적 할당의 구체적 과정]

미니배치 크기가 B일 때, OT-CFM은 다음 단계를 거친다:

Step 1: 비용 행렬(cost matrix) 계산. B개의 노이즈 $\{x_0^{(i)}\}$와 B개의 데이터 $\{x_1^{(j)}\}$ 사이의 모든 쌍별 거리를 계산한다:

$$C_{ij} = \|x_0^{(i)} - x_1^{(j)}\|^2, \quad i, j = 1, \ldots, B$$

이 비용 행렬 $C \in \mathbb{R}^{B \times B}$는 "노이즈 i를 데이터 j로 보내는 비용"을 나타낸다. 효율적인 계산을 위해 $\|a - b\|^2 = \|a\|^2 + \|b\|^2 - 2a^T b$ 공식을 사용하여 행렬 곱으로 한 번에 계산한다.

Step 2: 최적 할당(optimal assignment) 계산. 비용 행렬 C가 주어지면, 총 운반 비용 $\sum_{i=1}^{B} C_{i,\pi(i)}$를 최소화하는 순열(permutation) π를 찾는다:

$$\pi^* = \arg\min_{\pi \in S_B} \sum_{i=1}^{B} C_{i,\pi(i)}$$

이것은 고전적인 **할당 문제(assignment problem)**이며, **Hungarian algorithm**(또는 Kuhn-Munkres algorithm)으로 $O(B^3)$에 정확하게 풀 수 있다. Python에서는 `scipy.optimize.linear_sum_assignment`로 한 줄에 해결된다.

Step 3: 재페어링. 최적 순열 π^*에 따라 x_0를 재정렬한다: $x_0^{(i)}$를 $x_1^{(\pi^*(i))}$와 페어링한다. 이후는 표준 FM과 동일하게 직선 보간과 속도 예측을 수행한다.

9.2 OT-CFM이 필요한 이유

측면	표준 CFM	OT-CFM
(x_0, x_1) 페어링	랜덤 (독립 샘플)	**최적** (OT 기반)
경로 교차	많음	**적음**

학습 속도	보통	**빠름** (~2x 수렴 개선)
생성 품질	좋음	**더 좋음** (특히 적은 스텝에서)
추가 비용	없음	미니배치 내 OT 계산 ($O(B^3)$)

왜 OT 페어링이 학습을 빠르게 하는가? 랜덤 페어링에서는 서로 다른 학습 샘플이 같은 위치 ψ_t에서 모순된 속도를 요구할 수 있다 (경로 교차 때문에). 네트워크는 이 모순을 평균내어 처리해야 하므로, 손실 함수에 불필요한 분산(variance)이 추가된다. OT 페어링은 경로 교차를 줄여 이 분산을 감소시키므로, 같은 수의 학습 스텝에서 더 정확한 속도장을 학습할 수 있다.

Rectified Flow vs OT-CFM 비교: 둘 다 경로 교차를 줄이는 것이 목표이지만 접근 방식이 다르다:

- **Rectified Flow**: 학습 후 ODE를 풀어 (x_0, x_1) 쌍을 만들고 재학습 (offline, 추가 학습 라운드 필요)
- **OT-CFM**: 학습 중 매 미니배치에서 OT를 계산하여 페어링 최적화 (online, 단일 학습 라운드)

⚠️ **미니배치 OT의 한계**: 미니배치 내에서의 OT는 전체 분포 간 OT의 근사이다. 배치 크기가 클수록 더 정확한 근사가 된다. 실제로는 배치 크기 64~256 정도에서도 충분한 개선이 관찰된다. 또한, 미니배치 OT는 이론적으로 전체 OT에 수렴함이 증명되어 있다 (큰 수의 법칙에 의해, 미니배치 OT 결합의 기대값이 전체 OT 결합에 가까워진다).

---

10. 실습: Flow Matching vs DDPM 구현 비교

> 💡 **왜 구현 비교가 중요한가?**
>
> 이론적 차이를 코드로 확인하면 이해가 훨씬 깊어진다. 아래 코드를 비교하면 핵심 차이가 명확해진다: (1) DDPM은 노이즈 스케줄(`alpha_bar`)이 필요하지만 FM은 불필요, (2) DDPM은 노이즈 ϵ을 예측하지만 FM은 속도 $x_1 - x_0$을 예측, (3) FM의 생성 코드가 DDPM보다 훨씬 단순하다 (노이즈 추가 없이 Euler step만). 각 코드 블록의 주석을 주의 깊게 읽으면, 수학 공식과 코드의 일대일 대응을 확인할 수 있다.

10.1 학습 코드 비교

```python
# === DDPM 학습 ===
def ddpm_train_step(model, x_0, schedule):
    t = torch.randint(1, T+1, (B,))
    eps = torch.randn_like(x_0)
    alpha_bar = schedule.alpha_bar[t]
    x_t = sqrt(alpha_bar) * x_0 + sqrt(1-alpha_bar) * eps
    eps_pred = model(x_t, t)
    return F.mse_loss(eps_pred, eps)          # 노이즈 예측

# === Flow Matching 학습 ===
def fm_train_step(model, x_1):                # x_1 = 데이터
    x_0 = torch.randn_like(x_1)              # 노이즈
    t = torch.rand(B, 1, 1, 1)               # Uniform(0,1)
    psi_t = (1-t) * x_0 + t * x_1            # 직선 보간
    target = x_1 - x_0                        # 목표 속도
    v_pred = model(psi_t, t)
    return F.mse_loss(v_pred, target)         # 속도 예측
```

10.2 생성 코드 비교

```python
# === DDPM 생성 ===
x = torch.randn(B, C, H, W)                  # x_T
for t in reversed(range(1, T+1)):
    eps_pred = model(x, t)
    # 복잡한 업데이트: mean + noise
    x = 1/sqrt(alpha[t]) * (x - beta[t]/sqrt(1-alpha_bar[t]) * eps_
    if t > 1:
        x += sqrt(beta[t]) * torch.randn_like(x)  # 노이즈 추가!

# === Flow Matching 생성 ===
x = torch.randn(B, C, H, W)                  # x_0
N = 50                                         # 스텝 수
for i in range(N):
    t = torch.tensor(i / N)
    v = model(x, t)
    x = x + (1/N) * v                         # 단순 Euler step (노
```

> Flow Matching이 **구현도 단순하다**: Euler step 한 줄로 끝.

10.3 완전한 Flow Matching 모델 구현

아래는 간단한 시계열/이미지 생성을 위한 Flow Matching의 완전한 구현이다.

```python
import torch
import torch.nn as nn
import torch.nn.functional as F
import math

# ────────────────────────────────────────────────────────────
# 1) 시간 임베딩: 사인-코사인 위치 인코딩 (Transformer 스타일)
# ────────────────────────────────────────────────────────────
class SinusoidalTimeEmbedding(nn.Module):
    """
    스칼라 시간 t를 고차원 임베딩 벡터로 변환한다.
    Transformer의 위치 인코딩과 동일한 원리.
    dim: 임베딩 차원 (짝수여야 함)
    """
    def __init__(self, dim: int):
        super().__init__()
        self.dim = dim

    def forward(self, t: torch.Tensor) -> torch.Tensor:
        # t: (B,) 또는 (B,1) → (B, dim) 임베딩 벡터
        device = t.device
        half_dim = self.dim // 2
        # log-스케일 주파수 생성
        emb = math.log(10000.0) / (half_dim - 1)
        emb = torch.exp(torch.arange(half_dim, device=device) * -emb
        # t를 1차원으로 변환 후 외적으로 주파수 적용
        emb = t.view(-1, 1) * emb.view(1, -1)  # (B, half_dim)
        # sin, cos를 이어붙여 최종 임베딩 생성
        emb = torch.cat([torch.sin(emb), torch.cos(emb)], dim=-1)  #
        return emb


# ────────────────────────────────────────────────────────────
# 2) ResBlock: 시간 조건부 잔차 블록
# ────────────────────────────────────────────────────────────
class ResBlock(nn.Module):
    """
    시간 임베딩을 조건으로 받는 잔차 블록.
    GroupNorm → SiLU → Conv → (시간 임베딩 더하기) → GroupNorm → SiLU
    """
    def __init__(self, channels: int, time_emb_dim: int):
        super().__init__()
        # 첫 번째 정규화 + 활성화 + 컨볼루션
        self.norm1 = nn.GroupNorm(8, channels)
        self.conv1 = nn.Conv2d(channels, channels, 3, padding=1)
        # 시간 임베딩을 채널 차원으로 프로젝션
        self.time_proj = nn.Linear(time_emb_dim, channels)
        # 두 번째 정규화 + 활성화 + 컨볼루션
        self.norm2 = nn.GroupNorm(8, channels)
        self.conv2 = nn.Conv2d(channels, channels, 3, padding=1)

    def forward(self, x: torch.Tensor, t_emb: torch.Tensor) -> torch
```

```
        # x: (B, C, H, W), t_emb: (B, time_emb_dim)
        h = self.conv1(F.silu(self.norm1(x)))
        # 시간 임베딩을 공간 차원에 브로드캐스트하여 더하기
        h = h + self.time_proj(t_emb)[:, :, None, None]
        h = self.conv2(F.silu(self.norm2(h)))
        return x + h  # 잔차 연결


# ────────────────────────────────────────────────────────────
# 3) SimpleUNet: Flow Matching용 간단한 U-Net 아키텍처
# ────────────────────────────────────────────────────────────
class SimpleUNet(nn.Module):
    """
    Flow Matching을 위한 간단한 U-Net 속도장 예측 모델.
    입력: (x_t, t) → 출력: 예측 속도 v_θ(x_t, t)

    아키텍처:
    - Encoder: 다운샘플링 경로 (해상도 축소)
    - Bottleneck: 가장 낮은 해상도에서의 처리
    - Decoder: 업샘플링 경로 (해상도 복원) + skip connection
    """
    def __init__(self, in_channels: int = 3, base_channels: int = 6
                 time_emb_dim: int = 128):
        super().__init__()
        C = base_channels

        # ── 시간 임베딩 MLP ──
        self.time_embed = nn.Sequential(
            SinusoidalTimeEmbedding(time_emb_dim),
            nn.Linear(time_emb_dim, time_emb_dim),
            nn.SiLU(),
            nn.Linear(time_emb_dim, time_emb_dim),
        )

        # ── Encoder (다운샘플링 경로) ──
        self.conv_in = nn.Conv2d(in_channels, C, 3, padding=1)
        self.enc1 = ResBlock(C, time_emb_dim)         # 해상도 유지
        self.down1 = nn.Conv2d(C, C * 2, 4, stride=2, padding=1)  #
        self.enc2 = ResBlock(C * 2, time_emb_dim)     # 해상도 유지
        self.down2 = nn.Conv2d(C * 2, C * 4, 4, stride=2, padding=1

        # ── Bottleneck ──
        self.bottleneck = ResBlock(C * 4, time_emb_dim)

        # ── Decoder (업샘플링 경로) ──
        self.up2 = nn.ConvTranspose2d(C * 4, C * 2, 4, stride=2, pa
        self.dec2 = ResBlock(C * 4, time_emb_dim)  # skip connectio
        self.up1 = nn.ConvTranspose2d(C * 4, C, 4, stride=2, paddin
        self.dec1 = ResBlock(C * 2, time_emb_dim)  # skip connectio

        # ── 최종 출력 컨볼루션 ──
        self.conv_out = nn.Sequential(
```

```
        self.conv_out = nn.Sequential(
            nn.GroupNorm(8, C * 2),
            nn.SiLU(),
            nn.Conv2d(C * 2, in_channels, 3, padding=1),
        )

    def forward(self, x: torch.Tensor, t: torch.Tensor) -> torch.Te
        """
        x: (B, C_in, H, W) - 현재 시점의 샘플 ψ_t
        t: (B,) 또는 스칼라 - 현재 시간 [0, 1]
        반환: (B, C_in, H, W) - 예측 속도 v_θ(x, t)
        """
        # 시간 임베딩 계산
        t_emb = self.time_embed(t.view(-1))  # (B, time_emb_dim)

        # Encoder 경로
        h = self.conv_in(x)           # (B, C, H, W)
        h1 = self.enc1(h, t_emb)      # (B, C, H, W) — skip connect
        h = self.down1(h1)            # (B, 2C, H/2, W/2)
        h2 = self.enc2(h, t_emb)      # (B, 2C, H/2, W/2) — skip co
        h = self.down2(h2)            # (B, 4C, H/4, W/4)

        # Bottleneck
        h = self.bottleneck(h, t_emb) # (B, 4C, H/4, W/4)

        # Decoder 경로 (skip connection 결합)
        h = self.up2(h)               # (B, 2C, H/2, W/2)
        h = torch.cat([h, h2], dim=1) # (B, 4C, H/2, W/2) — skip co
        h = self.dec2(h, t_emb)       # (B, 4C, H/2, W/2)
        h = self.up1(h)               # (B, C, H, W)
        h = torch.cat([h, h1], dim=1) # (B, 2C, H, W) — skip connec
        h = self.dec1(h, t_emb)       # (B, 2C, H, W)

        # 최종 출력: 예측 속도
        v_pred = self.conv_out(h)     # (B, C_in, H, W)
        return v_pred


# ────────────────────────────────────────────────────────────
# 4) FlowMatchingTrainer: Flow Matching 학습 래퍼 클래스
# ────────────────────────────────────────────────────────────
class FlowMatchingTrainer:
    """
    Flow Matching 학습을 관리하는 클래스.
    직선 보간 경로 ψ_t = (1-t)*x_0 + t*x_1 을 사용하고,
    목표 속도 u_t = x_1 - x_0 을 예측하도록 학습한다.
    """
    def __init__(self, model: nn.Module, lr: float = 1e-4,
                 device: str = 'cuda'):
        self.model = model.to(device)
        self.device = device
        self.optimizer = torch.optim.AdamW(model.parameters(), lr=lr
```

```python
def train_step(self, x_1: torch.Tensor) -> float:
    """
    한 번의 학습 스텝 수행.

    x_1: (B, C, H, W) - 실제 데이터 (목표 분포에서 샘플)

    반환: loss 값 (float)
    """
    self.model.train()
    B = x_1.shape[0]
    x_1 = x_1.to(self.device)

    # 1) 사전 분포에서 노이즈 샘플링: x_0 ~ N(0, I)
    x_0 = torch.randn_like(x_1)

    # 2) 시간 샘플링: t ~ Uniform(0, 1)
    #    shape을 (B, 1, 1, 1)로 만들어 이미지 차원에 브로드캐스트
    t = torch.rand(B, 1, 1, 1, device=self.device)

    # 3) 직선 보간으로 중간 샘플 생성: ψ_t = (1-t)*x_0 + t*x_1
    psi_t = (1 - t) * x_0 + t * x_1

    # 4) 목표 속도 계산: u_t = x_1 - x_0 (시간에 무관한 상수)
    target_velocity = x_1 - x_0

    # 5) 모델이 속도 예측: v_ϑ(ψ_t, t)
    #    t를 (B,) 스칼라로 변환하여 모델에 전달
    v_pred = self.model(psi_t, t.squeeze())

    # 6) CFM 손실 계산: L = E[||v_ϑ(ψ_t, t) - (x_1 - x_0)||²]
    loss = F.mse_loss(v_pred, target_velocity)

    # 7) 역전파 및 파라미터 업데이트
    self.optimizer.zero_grad()
    loss.backward()
    # 그래디언트 클리핑으로 학습 안정화
    torch.nn.utils.clip_grad_norm_(self.model.parameters(), max_
    self.optimizer.step()

    return loss.item()

@torch.no_grad()
def sample_euler(self, shape: tuple, num_steps: int = 50) -> tor
    """
    Euler method로 ODE를 풀어 샘플 생성.

    shape: 생성할 텐서의 shape, 예: (B, 3, 32, 32)
    num_steps: ODE 솔버 스텝 수 (많을수록 정확하지만 느림)

    반환: 생성된 샘플 (B, C, H, W)
    """
```

```
        self.model.eval()

        # 노이즈에서 시작: x_0 ~ N(0, I)
        x = torch.randn(shape, device=self.device)
        dt = 1.0 / num_steps  # 스텝 크기 h = 1/N

        for i in range(num_steps):
            t_scalar = i / num_steps
            # 배치 내 모든 샘플에 동일한 시간 적용
            t = torch.full((shape[0],), t_scalar, device=self.device
            # 속도 예측
            v = self.model(x, t)
            # Euler 업데이트: x_{t+h} = x_t + h * v_θ(x_t, t)
            x = x + dt * v

        return x  # 생성된 이미지 (t=1에서의 샘플)
```

10.4 OT-CFM (Optimal Transport Conditional Flow Matching) 구현

```python
import torch
from scipy.optimize import linear_sum_assignment

# ────────────────────────────────────────────────────────────
# OT-CFM: 미니배치 내 Optimal Transport로 (x_0, x_1) 페어링 최적화
# ────────────────────────────────────────────────────────────

def compute_ot_plan(x_0: torch.Tensor, x_1: torch.Tensor) -> torch.T
    """
    미니배치 내에서 Optimal Transport 매핑을 계산한다.
    x_0와 x_1 사이의 L2 거리 행렬을 기반으로 Hungarian algorithm을 사용

    x_0: (B, C, H, W) - 노이즈 샘플들
    x_1: (B, C, H, W) - 데이터 샘플들

    반환: x_0를 재정렬한 텐서 (B, C, H, W)
          → x_0[i]가 x_1[i]에 대한 OT 최적 매핑이 되도록 정렬됨
    """
    B = x_0.shape[0]

    # 각 샘플을 1차원 벡터로 펼침 (거리 계산용)
    x_0_flat = x_0.view(B, -1)  # (B, C*H*W)
    x_1_flat = x_1.view(B, -1)  # (B, C*H*W)

    # L2 비용 행렬 계산: C[i,j] = ||x_0[i] - x_1[j]||²
    # 효율적 계산: ||a-b||² = ||a||² + ||b||² - 2<a,b>
    cost_matrix = (
        torch.sum(x_0_flat ** 2, dim=1, keepdim=True)       # (B, 1,
        + torch.sum(x_1_flat ** 2, dim=1, keepdim=True).T   # (1, B,
        - 2 * x_0_flat @ x_1_flat.T                          # (B, 
    )

    # Hungarian algorithm으로 최적 할당 계산
    # 시간 복잡도: O(B³) — 미니배치 크기에 큐빅하므로 B가 너무 크면 느리
    row_indices, col_indices = linear_sum_assignment(cost_matrix.cpu

    # x_0를 OT 최적 순서로 재정렬
    # col_indices[i]는 x_0[i]가 매핑되어야 할 x_1의 인덱스
    # 따라서 x_0[col_indices]가 x_1과 최적으로 매핑된 노이즈가 됨
    return x_0[col_indices]


def ot_cfm_train_step(model, x_1, optimizer, device='cuda'):
    """
    OT-CFM 학습 스텝.
    표준 CFM과의 유일한 차이: x_0와 x_1의 페어링을 OT로 최적화.

    x_1: (B, C, H, W) - 실제 데이터
    """
    model.train()
    B = x_1.shape[0]
    x_1 = x_1.to(device)
```

```
    # 1) 노이즈 샘플링 (표준 CFM과 동일)
    x_0 = torch.randn_like(x_1)

    # 2) ★ OT 최적 페어링 계산 ★
    #    x_0를 재정렬하여 x_0[i]와 x_1[i]가 OT 최적 쌍이 되도록 함
    x_0 = compute_ot_plan(x_0, x_1)

    # 3) 이후는 표준 CFM과 동일
    t = torch.rand(B, 1, 1, 1, device=device)
    psi_t = (1 - t) * x_0 + t * x_1        # 직선 보간
    target = x_1 - x_0                       # 목표 속도

    v_pred = model(psi_t, t.squeeze())
    loss = F.mse_loss(v_pred, target)

    optimizer.zero_grad()
    loss.backward()
    torch.nn.utils.clip_grad_norm_(model.parameters(), max_norm=1.0
    optimizer.step()

    return loss.item()


# ────────────────────────────────────────────────────────────
# 전체 학습 루프 예시 (OT-CFM)
# ────────────────────────────────────────────────────────────

def train_ot_cfm(model, dataloader, epochs=100, device='cuda'):
    """
    OT-CFM 전체 학습 루프.

    model: SimpleUNet 또는 다른 속도장 예측 모델
    dataloader: 학습 데이터 로더
    epochs: 학습 에폭 수
    """
    optimizer = torch.optim.AdamW(model.parameters(), lr=1e-4)
    # 학습률 스케줄러: 코사인 어닐링으로 점차 감소
    scheduler = torch.optim.lr_scheduler.CosineAnnealingLR(
        optimizer, T_max=epochs
    )

    for epoch in range(epochs):
        total_loss = 0.0
        num_batches = 0

        for batch in dataloader:
            x_1 = batch[0]  # 데이터 (이미지 등)
            loss = ot_cfm_train_step(model, x_1, optimizer, device)
            total_loss += loss
            num_batches += 1
```

```
        avg_loss = total_loss / num_batches
        scheduler.step()

        if (epoch + 1) % 10 == 0:
            print(f"Epoch {epoch+1}/{epochs}, "
                  f"Loss: {avg_loss:.4f}, "
                  f"LR: {scheduler.get_last_lr()[0]:.6f}")
```

10.5 고차 ODE 솔버: Heun's Method 및 RK4

Euler method는 1차 정확도만 가지므로, 적은 스텝으로 더 정확한 샘플링이 필요할 때 고차 솔버를 사용한다.

```python
import torch
import torch.nn as nn

# ────────────────────────────────────────────────────────────
# Heun's Method (Improved Euler / 2차 Runge-Kutta)
# ────────────────────────────────────────────────────────────

@torch.no_grad()
def sample_heun(model: nn.Module, shape: tuple,
                num_steps: int = 25, device: str = 'cuda') -> torch
    """
    Heun's method (2차 정확도)로 ODE를 풀어 샘플 생성.

    Euler는 현재 점에서의 기울기만 사용하지만,
    Heun은 현재 점과 다음 점에서의 기울기를 **평균**하여 사용한다.

    정확도: O(h²) — Euler의 O(h)보다 한 차수 높음
    비용: 스텝당 모델 호출 2번 (Euler의 2배)
    → 같은 NFE(Number of Function Evaluations) 대비 Euler보다 정확

    shape: 생성할 텐서의 shape, 예: (B, 3, 32, 32)
    num_steps: ODE 솔버 스텝 수
    """
    model.eval()
    x = torch.randn(shape, device=device)   # x_0 ~ N(0, I)
    dt = 1.0 / num_steps

    for i in range(num_steps):
        t_cur = i / num_steps
        t_next = (i + 1) / num_steps

        # 현재 시점 t에서의 배치 시간 텐서 생성
        t_batch_cur = torch.full((shape[0],), t_cur, device=device)
        t_batch_next = torch.full((shape[0],), t_next, device=devic

        # ── 1단계: Euler 예측 (predictor) ──
        v_cur = model(x, t_batch_cur)               # 현재 점에서의
        x_euler = x + dt * v_cur                     # Euler로 다음

        # ── 2단계: 보정 (corrector) ──
        v_next = model(x_euler, t_batch_next)        # 예측된 다음 ㅈ

        # ── 두 속도의 평균으로 최종 업데이트 ──
        # x_{t+h} = x_t + h * (v(x_t, t) + v(x̂_{t+h}, t+h)) / 2
        x = x + dt * (v_cur + v_next) / 2

    return x


# ────────────────────────────────────────────────────────────
# RK4 (4차 Runge-Kutta) — 가장 고전적이고 정확한 고정 스텝 솔버
# ────────────────────────────────────────────────────────────
```

```python
@torch.no_grad()
def sample_rk4(model: nn.Module, shape: tuple,
               num_steps: int = 15, device: str = 'cuda') -> torch.
    """
    4차 Runge-Kutta method로 ODE를 풀어 샘플 생성.

    RK4는 4개의 기울기 평가를 가중 평균하여 매우 높은 정확도를 달성한다:
    - k1: 구간 시작점에서의 기울기
    - k2: k1을 사용한 중간점에서의 기울기
    - k3: k2를 사용한 중간점에서의 기울기
    - k4: k3을 사용한 구간 끝점에서의 기울기

    정확도: O(h⁴) — 매우 높은 정확도
    비용: 스텝당 모델 호출 4번
    → 적은 스텝(10~15)으로도 매우 정확한 샘플링 가능

    shape: 생성할 텐서의 shape, 예: (B, 3, 32, 32)
    num_steps: ODE 솔버 스텝 수
    """
    model.eval()
    x = torch.randn(shape, device=device)  # x_0 ~ N(0, I)
    dt = 1.0 / num_steps

    for i in range(num_steps):
        t_cur = i / num_steps
        t_mid = (i + 0.5) / num_steps     # 중간 시점
        t_next = (i + 1) / num_steps       # 다음 시점

        # 배치 시간 텐서
        t_batch_cur = torch.full((shape[0],), t_cur, device=device)
        t_batch_mid = torch.full((shape[0],), t_mid, device=device)
        t_batch_next = torch.full((shape[0],), t_next, device=devic

        # ── k1: 현재 점에서의 기울기 ──
        k1 = model(x, t_batch_cur)

        # ── k2: k1으로 중간점까지 이동한 후의 기울기 ──
        k2 = model(x + 0.5 * dt * k1, t_batch_mid)

        # ── k3: k2로 중간점까지 이동한 후의 기울기 (k2와 다른 점!) ──
        k3 = model(x + 0.5 * dt * k2, t_batch_mid)

        # ── k4: k3으로 끝점까지 이동한 후의 기울기 ──
        k4 = model(x + dt * k3, t_batch_next)

        # ── 가중 평균으로 최종 업데이트 ──
        # x_{t+h} = x_t + (h/6) * (k1 + 2*k2 + 2*k3 + k4)
        x = x + (dt / 6.0) * (k1 + 2 * k2 + 2 * k3 + k4)

    return x
```

```python
# ────────────────────────────────────────────────────────────
# 솔버 비교 유틸리티
# ────────────────────────────────────────────────────────────

def compare_solvers(model, shape, device='cuda'):
    """
    세 가지 ODE 솔버의 성능을 비교하는 유틸리티 함수.

    동일한 NFE (Number of Function Evaluations) 기준으로 비교:
    - Euler 100 스텝 = 100 NFE
    - Heun 50 스텝 = 100 NFE (스텝당 2번 호출)
    - RK4 25 스텝 = 100 NFE (스텝당 4번 호출)
    """
    import time

    results = {}
    nfe_target = 100  # 총 모델 호출 수를 동일하게 맞춤

    configs = [
        ("Euler", sample_euler_simple, nfe_target),       # 100 스텝
        ("Heun",  sample_heun,         nfe_target // 2),  # 50 스텝
        ("RK4",   sample_rk4,          nfe_target // 4),  # 25 스텝
    ]

    for name, solver_fn, steps in configs:
        start = time.time()
        samples = solver_fn(model, shape, num_steps=steps, device=de
        elapsed = time.time() - start
        results[name] = {
            'samples': samples,
            'steps': steps,
            'nfe': steps * (1 if 'Euler' in name else 2 if 'Heun' in
            'time': elapsed,
        }
        print(f"{name}: {steps} 스텝, {results[name]['nfe']} NFE, "
              f"{elapsed:.2f}초")

    return results


def sample_euler_simple(model, shape, num_steps=50, device='cuda'):
    """비교용 Euler 샘플링 (독립 함수 버전)."""
    model.eval()
    x = torch.randn(shape, device=device)
    dt = 1.0 / num_steps
    for i in range(num_steps):
        t = torch.full((shape[0],), i / num_steps, device=device)
        v = model(x, t)
        x = x + dt * v
    return x
```

⚠️ **실전 팁**: 같은 NFE(모델 호출 횟수) 기준으로, 일반적으로 **Heun > Euler**, **RK4 > Heun** 순으로 정확도가 높다. 하지만 Flow Matching의 직선 경로는 Euler만으로도 상당히 정확하므로, 실무에서는 Euler 50스텝 또는 Heun 25스텝이 일반적이다.

10.6 Flow Matching에서의 CFG 구현

```python
import torch
import torch.nn as nn
import torch.nn.functional as F

# ────────────────────────────────────────────────────────────
# 조건부 Flow Matching 모델 (CFG 지원)
# ────────────────────────────────────────────────────────────

class ConditionalFlowMatchingModel(nn.Module):
    """
    클래스 조건부 Flow Matching 모델.
    Classifier-Free Guidance (CFG) 학습을 지원한다.

    학습 시 일정 확률(p_uncond)로 클래스 조건을 드롭하여,
    동일 모델이 조건부/비조건부 속도장을 모두 학습한다.
    """
    def __init__(self, base_model: nn.Module, num_classes: int,
                 class_emb_dim: int = 128, p_uncond: float = 0.1):
        """
        base_model: 기본 U-Net 모델 (시간 + 클래스 임베딩을 받는 구조)
        num_classes: 클래스 수
        class_emb_dim: 클래스 임베딩 차원
        p_uncond: 비조건부 학습 확률 (CFG를 위한 드롭아웃 비율)
        """
        super().__init__()
        self.base_model = base_model
        self.p_uncond = p_uncond

        # 클래스 임베딩 테이블 (num_classes + 1: 마지막은 "조건 없음"
        self.class_embedding = nn.Embedding(num_classes + 1, class_
        self.null_class_idx = num_classes  # "조건 없음" 인덱스

    def forward(self, x: torch.Tensor, t: torch.Tensor,
                class_label: torch.Tensor = None) -> torch.Tensor:
        """
        x: (B, C, H, W) - 현재 시점의 샘플
        t: (B,) - 시간
        class_label: (B,) - 클래스 레이블 (None이면 비조건부)

        반환: 예측 속도 v_θ(x, t, c)
        """
        B = x.shape[0]

        if class_label is None:
            # 비조건부: null 클래스 사용
            class_label = torch.full(
                (B,), self.null_class_idx,
                device=x.device, dtype=torch.long
            )

        # 학습 모드에서 CFG 드롭아웃 적용
        if self.training:
```

```
            # 확률 p_uncond로 클래스 조건을 null로 교체
            drop_mask = torch.rand(B, device=x.device) < self.p_unc
            class_label = torch.where(
                drop_mask,
                torch.full_like(class_label, self.null_class_idx),
                class_label
            )

        # 클래스 임베딩 계산
        c_emb = self.class_embedding(class_label)  # (B, class_emb_

        # 기본 모델에 시간과 클래스 임베딩을 함께 전달
        # (실제 구현에서는 base_model이 c_emb를 받을 수 있도록 수정 필
        v_pred = self.base_model(x, t, c_emb)
        return v_pred


# ────────────────────────────────────────────────────────────
# CFG가 적용된 학습 루프
# ────────────────────────────────────────────────────────────

def cfg_fm_train_step(model, x_1, class_labels, optimizer, device='
    """
    CFG를 지원하는 Flow Matching 학습 스텝.

    model: ConditionalFlowMatchingModel
    x_1: (B, C, H, W) - 실제 데이터
    class_labels: (B,) - 클래스 레이블
    """
    model.train()
    B = x_1.shape[0]
    x_1 = x_1.to(device)
    class_labels = class_labels.to(device)

    # 표준 CFM과 동일한 절차 (모델이 내부에서 CFG 드롭아웃 처리)
    x_0 = torch.randn_like(x_1)
    t = torch.rand(B, 1, 1, 1, device=device)
    psi_t = (1 - t) * x_0 + t * x_1
    target = x_1 - x_0

    # 모델은 training=True일 때 자동으로 p_uncond 확률로 조건 드롭
    v_pred = model(psi_t, t.squeeze(), class_labels)
    loss = F.mse_loss(v_pred, target)

    optimizer.zero_grad()
    loss.backward()
    torch.nn.utils.clip_grad_norm_(model.parameters(), max_norm=1.0
    optimizer.step()

    return loss.item()
```

```python
# ────────────────────────────────────────────────────────────
# CFG가 적용된 샘플링 (Euler method)
# ────────────────────────────────────────────────────────────

@torch.no_grad()
def sample_with_cfg(model, shape: tuple, class_label: int,
                    guidance_scale: float = 7.5,
                    num_steps: int = 50,
                    device: str = 'cuda') -> torch.Tensor:
    """
    Classifier-Free Guidance가 적용된 Flow Matching 샘플링.

    CFG 공식:
        ṽ_θ(x, t, c) = v_θ(x, t, ∅) + w * (v_θ(x, t, c) - v_θ(x, t,

    여기서:
    - v_θ(x, t, c): 조건부 속도 (클래스 c가 주어진 경우)
    - v_θ(x, t, ∅): 비조건부 속도 (조건 없이)
    - w: guidance_scale (가이던스 강도, 보통 3.0 ~ 10.0)

    shape: 생성할 텐서의 shape, 예: (B, 3, 32, 32)
    class_label: 생성할 클래스
    guidance_scale: CFG 가이던스 강도 (w)
                    w=1.0이면 표준 조건부 생성
                    w>1.0이면 더 강한 조건부 생성 (품질↑, 다양성↓)
    num_steps: ODE 솔버 스텝 수
    """
    model.eval()
    B = shape[0]

    # 노이즈에서 시작
    x = torch.randn(shape, device=device)
    dt = 1.0 / num_steps

    # 클래스 레이블 텐서 준비
    cond_labels = torch.full((B,), class_label, device=device, dtype
    # null 클래스 (비조건부)
    null_labels = torch.full(
        (B,), model.null_class_idx, device=device, dtype=torch.long
    )

    for i in range(num_steps):
        t = torch.full((B,), i / num_steps, device=device)

        # ── 조건부 속도 예측 ──
        v_cond = model(x, t, cond_labels)    # v_θ(x, t, c)

        # ── 비조건부 속도 예측 ──
        v_uncond = model(x, t, null_labels)  # v_θ(x, t, ∅)

        # ── CFG 적용 ──
```

```
        # ṽ = v_uncond + w * (v_cond - v_uncond)
        #   = (1 - w) * v_uncond + w * v_cond
        v_guided = v_uncond + guidance_scale * (v_cond - v_uncond)

        # ── Euler 업데이트 ──
        x = x + dt * v_guided

    return x

    # ⚠️ 참고: CFG는 스텝당 모델을 2번 호출하므로 (조건부 + 비조건부),
    #          총 NFE = 2 * num_steps 이다.
    #          이것이 FLUX에서 Guidance Distillation을 사용하는 이유이
```

⚠️ **CFG guidance_scale 선택 가이드**:

- `w = 1.0`: 표준 조건부 생성 (CFG 없음과 동일)
- `w = 3.0 ~ 5.0`: 적당한 가이던스 (다양성과 품질의 균형)
- `w = 7.5`: 일반적인 기본값 (Stable Diffusion 등에서 사용)
- `w > 10.0`: 매우 강한 가이던스 (과포화/아티팩트 위험)

---

11. 핵심 수식 요약

> 이 표는 강의 전체의 핵심을 한 페이지로 압축한 것이다. 시험/복습 시 이 표만 보면 전체 흐름을 떠올릴 수 있도록 구성했다. 각 수식이 어느 섹션에서 유도되었는지 참조를 함께 표시한다.

이름	수식	참조
CNF ODE	$dx_t/dt = v_\theta(x_t, t)$	섹션 1.1
직선 보간	$\psi_t = (1-t)x_0 + tx_1$	섹션 2.3
목표 속도	$u_t = x_1 - x_0$ (시간에 무관한 상수)	섹션 2.3
CFM Loss	$\mathcal{L} = \mathbb{E}_{t,x_0,x_1}[\lvert v_\theta(\psi_t, t) - (x_1 - x_0)\rvert^2]$	섹션 2.4
핵심 정리	$\nabla_\theta \mathcal{L}_{\text{CFM}} = \nabla_\theta \mathcal{L}_{\text{FM}}$ (전기대값의 법칙)	섹션 2.4
Euler Sampling	$x_{t+h} = x_t + h \cdot v_\theta(x_t, t)$ — 오차 $O(h)$	섹션 3.2
Heun Sampling	$x_{t+h} = x_t + \frac{h}{2}(v(x_t, t) + v(\hat{x}_{t+h}, t+h))$ — 오차 $O(h^2)$	섹션 10.5
RK4 Sampling	$x_{t+h} = x_t + \frac{h}{6}(k_1 + 2k_2 + 2k_3 + k_4)$ — 오차 $O(h^4)$	섹션 10.5
가우시안 경로	$\psi_t = \alpha_t x_1 + \sigma_t x_0$ (DDPM과 동치)	섹션 4.2

속도-노이즈 변환	$v_t = \frac{\alpha_t'}{\alpha_t}\psi_t + (\sigma_t' - \frac{\alpha_t'\sigma_t}{\alpha_t})\epsilon$	섹션 4.3
FM-CFG	$\tilde{v} = v(x_t, \varnothing) + w(v(x_t, c) - v(x_t, \varnothing))$	섹션 8.1
연속 방정식	$\partial_t p_t + \nabla \cdot (p_t v_t) = 0$	섹션 7.3
Fokker-Planck	$\partial_t p_t = -\nabla \cdot (v_t p_t) + \frac{\sigma_t^2}{2}\nabla^2 p_t$	섹션 7.3
조건부 스코어	$\nabla \log p_t(x \mid x_1) = -(x - \alpha_t x_1)/\sigma_t^2$	섹션 7.2
OT 비용 행렬	$C_{ij} = \lvert x_0^{(i)} - x_1^{(j)} \rvert^2$	섹션 9.1

12. 다음 강의 미리보기

다음 강의에서는:

- **DiT (Diffusion Transformer)**: U-Net을 대체하는 Transformer 아키텍처. Flow Matching과 결합하면 SD3의 MM-DiT가 된다. U-Net 대비 scaling 특성이 우수하여 (파라미터 수를 늘릴수록 성능이 더 안정적으로 향상), 대규모 모델에 적합하다.
- **Consistency Models**: 1~4 스텝으로 생성하는 가속 기법. Rectified Flow의 Reflow가 "경로를 직선화하여 1-step 생성"을 가능하게 하는 것과 유사한 목표를 가지지만, 접근 방식이 다르다. Consistency Model은 ODE 궤적의 모든 점이 동일한 끝점으로 수렴하도록 학습하여, 중간 지점에서도 최종 결과를 즉시 예측할 수 있게 한다.
- **VAR (Visual Autoregressive)**: Kaiming He의 NeurIPS 2024 Best Paper. Diffusion/Flow Matching과 완전히 다른 패러다임으로, 이미지를 "해상도 단계별로 순차 생성"하는 autoregressive 방식이다. Flow Matching의 연속 시간 ODE 접근과 대비되는 이산적 접근이다.

> 💡 **이 강의와의 연결**: 다음 강의의 모든 주제는 이번 강의에서 배운 Flow Matching을 기반으로 한다. DiT는 FM의 속도장 v_θ를 Transformer로 파라미터화한 것이고, Consistency Model은 FM의 ODE 궤적 위에서 정의되며, VAR조차 FM과의 이론적 연결점(연속 vs 이산)을 이해해야 장단점을 비교할 수 있다.

13. 참고문헌 (References)

핵심 Flow Matching 논문

1. 📄 **Lipman, Y., Chen, R. T. Q., Ben-Hamu, H., Nickel, M., & Le, M.** (2023). "Flow Matching for Generative Modeling." *International Conference on Learning Representations (ICLR)*.

2. **Albergo, M. S., & Vanden-Eijnden, E.** (2023). "Building Normalizing Flows with Stochastic Interpolants." *International Conference on Learning Representations (ICLR)*.

3. **Liu, X., Gong, C., & Liu, Q.** (2023). "Flow Straight and Fast: Learning to Generate and Transfer Data with Rectified Flow." *International Conference on Learning Representations (ICLR)*.

4. **Tong, A., Malkin, N., Huguet, G., Zhang, Y., Rector-Brooks, J., Fatras, K., Wolf, G., & Bengio, Y.** (2024). "Improving and Generalizing Flow-Based Generative Models with Minibatch Optimal Transport." *Transactions on Machine Learning Research (TMLR)*.

응용 및 대규모 모델

5. **Esser, P., Kulal, S., Blattmann, A., Entezari, R., Muller, J., Saini, H., Levi, Y., Lorber, D., & Rombach, R.** (2024). "Scaling Rectified Flow Transformers for High-Resolution Image Synthesis." *International Conference on Machine Learning (ICML)*. (Stable Diffusion 3)

6. **Black Forest Labs** (2024). "FLUX." (Guidance Distillation을 적용한 Flow Matching 기반 이미지 생성 모델)

7. **Polyak, A., et al.** (2024). "Movie Gen: A Cast of Media Foundation Models." *Meta AI*. (Flow Matching 기반 비디오 생성)

배경 이론

8. **Chen, R. T. Q., Rubanova, Y., Bettencourt, J., & Duvenaud, D.** (2018). "Neural Ordinary Differential Equations." *Advances in Neural Information Processing Systems (NeurIPS)*. (Neural ODE 원 논문)

9. **Ho, J., Jain, A., & Abbeel, P.** (2020). "Denoising Diffusion Probabilistic Models." *Advances in Neural Information Processing Systems (NeurIPS)*. (DDPM 원 논문)

10. **Song, Y., Sohl-Dickstein, J., Kingma, D. P., Kumar, A., Ermon, S., & Poole, B.** (2021). "Score-Based Generative Modeling through Stochastic Differential Equations." *International Conference on Learning Representations (ICLR)*. (Score SDE 프레임워크)

강의 자료

11. MIT 6.S184: "Introduction to Flow Matching and Diffusion Models" (2025, 2026). *Massachusetts Institute of Technology*.